Spring 2025 Homework Modeling Exercises Weeks 1 & 3

Group 2: group names, University of Chicago

03-31-2026

Download data

3. Set RStudio's working directory to the folder containing the datasets for this homework and open the PJM data set ("data_PJM.csv"), as below. This dataset is constructed from hourly pricing data from PJM's Data Miner 2 and includes both day-ahead ("DA") and real-time("RT") locational marginal pricing ("LMP") data.

```
data_PJM <- fread(paste0(wd,"/", "PJM Data.csv"))
```

4. Load the following data set, which combines pricing and weather data for Chicago, Milwaukee, and Corpus Christi.

```
data_Chicago <- fread(paste0(wd,"/",'Data Chicago.csv'))
data_Milwaukee <- fread(paste0(wd,"/",'Data Milwaukee.csv'))
data_CorpusChristi<- fread(paste0(wd,"/",'Data Corpus Christi.csv'))
data_Austin <- fread(paste0(wd,"/",'Data Austin.csv'))
```

III. Data Familiarization

5. Let's explore the PJM data! The following code identifies the fields of the dataset

```
names(data_PJM)
```

```
##  [1] "datetime_beginning_utc" "datetime_beginning_ept" "pnode_id"
##  [4] "pnode_name"             "voltage"                "equipment"
##  [7] "type"                   "zone"                   "system_energy_price_rt"
## [10] "total_lmp_rt"           "congestion_price_rt"    "marginal_loss_price_rt"
## [13] "system_energy_price_da" "total_lmp_da"           "congestion_price_da"
## [16] "marginal_loss_price_da" "spread"                 "date"
## [19] "hour"                   "am_pm"                  "day"
## [22] "day_of_year"            "month_num"              "month"
## [25] "year"                   "day_of_week"            "weekend"
## [28] "isHoliday"              "holiday"
```

6. To see the first few observations of the data, run the code below. What is the unit ofobservation?

```
head(data_PJM)
```

```
##    datetime_beginning_utc datetime_beginning_ept pnode_id pnode_name voltage
##                    <char>                 <char>    <i64>     <char>  <lgcl>
## 1:    2/1/2021 4:00:00 AM  1/31/2021 11:00:00 PM        1        PJM      NA
## 2:    2/1/2021 4:00:00 AM  1/31/2021 11:00:00 PM    51205  BRANDONSH      NA
## 3:    2/1/2021 4:00:00 AM  1/31/2021 11:00:00 PM    51206  BRUNSWICK      NA
## 4:    2/1/2021 4:00:00 AM  1/31/2021 11:00:00 PM    51211  COOKSTOWN      NA
## 5:    2/1/2021 4:00:00 AM  1/31/2021 11:00:00 PM    51214      DOVER      NA
## 6:    2/1/2021 4:00:00 AM  1/31/2021 11:00:00 PM    51215  DPL NORTH      NA
```

```
##    equipment      type   zone system_energy_price_rt total_lmp_rt
##       <lgcl>    <char> <char>                  <num>        <num>
## 1:        NA      ZONE                         25.06     25.03782
## 2:        NA AGGREGATE                          25.06     25.40750
## 3:        NA AGGREGATE                          25.06     21.82667
## 4:        NA AGGREGATE                          25.06     18.05833
## 5:        NA AGGREGATE                          25.06     20.76083
## 6:        NA AGGREGATE                          25.06     20.29252
##    congestion_price_rt marginal_loss_price_rt system_energy_price_da
##                  <num>                  <num>                  <num>
## 1:           -0.037294               0.015118                  21.31
## 2:           -1.106667               1.454167                  21.31
## 3:           -4.904167               1.670833                  21.31
## 4:           -8.830000               1.828333                  21.31
## 5:           -6.325000               2.025833                  21.31
## 6:           -6.316337               1.548852                  21.31
##    total_lmp_da congestion_price_da marginal_loss_price_da    spread       date
##           <num>               <num>                  <num>     <num>     <IDat>
## 1:     21.35672           -0.205410               0.252130 -3.681104 2021-01-31
## 2:     21.16000           -1.640000               1.490000 -4.247500 2021-01-31
## 3:     21.58000           -1.810000               2.080000 -0.246667 2021-01-31
## 4:     19.94000           -3.590000               2.220000  1.881667 2021-01-31
## 5:     20.91000           -2.750000               2.350000  0.149167 2021-01-31
## 6:     20.49766           -2.755891               1.943547  0.205141 2021-01-31
##     hour  am_pm   day day_of_year month_num  month  year day_of_week weekend
##    <int> <char> <int>       <int>     <int> <char> <int>      <char>  <lgcl>
## 1:    23     PM    31          31         1    Jan  2021      Sunday    TRUE
## 2:    23     PM    31          31         1    Jan  2021      Sunday    TRUE
## 3:    23     PM    31          31         1    Jan  2021      Sunday    TRUE
## 4:    23     PM    31          31         1    Jan  2021      Sunday    TRUE
## 5:    23     PM    31          31         1    Jan  2021      Sunday    TRUE
## 6:    23     PM    31          31         1    Jan  2021      Sunday    TRUE
##    isHoliday holiday
##       <lgcl>  <char>
## 1:     FALSE
## 2:     FALSE
## 3:     FALSE
## 4:     FALSE
## 5:     FALSE
## 6:     FALSE
```

7. The code below identifies the final date of the PJM data as December, 12, 2023. How wouldyou identify the initial date?

```r
max(data_PJM$date)
```

```
## [1] "2023-12-31"
```

8. The code below identifies the set of pricing node types in the PJM data and calculates how many observations correspond to each type. How would you do the same for pricing `nodesnames`?

```r
# unique(data_PJM$type)
# data_PJM[,.N, by = type]
```

9. The code below identifies the number of unique holiday values. How would you identify thenumber of observations falling on holidays? How would you identify the number of uniquedifferent pricing nodes?

```r
length(unique(data_PJM$holiday))
```

```
## [1] 12
```

10. The following code restricts the PJM data to only the "date" and "total_lmp_rt" fields. Whatfields would be interesting to study in conjunction?

```r
# data_PJM[, .(date, total_lmp_rt)]
```

11. The code below creates a new dataset, restricted to PJM observations falling on Christmas.Create a dataset, named "holidays_PJM," consisting of all observations falling on a holiday.

```r
# xmas_PJM <- data_PJM[holiday == 'Christmas']
```

12. The code below calculates the average RT LMP on holidays in November and December, during AM hours, for the "'CHICAGO GEN HUB" pricing node.6 What observations arebeing selected? What is being averaged over? What stands out about the results? Write codethat averages DA LMP within each month, for the "Chicago Gen Hub." Formulate ahypothesis based on the result you observe. How could you check your hypothesis?

```r
# holidays_PJM[month == 'Dec' &
#                pnode_name == 'CHICAGO GEN HUB' &
#                am_pm == 'AM',
#   .(average_lmp_rt = mean(total_lmp_rt)), by = .(hour, year)] %>%
#   .[order(year)]
```

IV. Introduction to Plotting

13. Create a dataset called "data_CGH," restricted to the "'CHICAGO GEN HUB." What does the number of observations reflect?

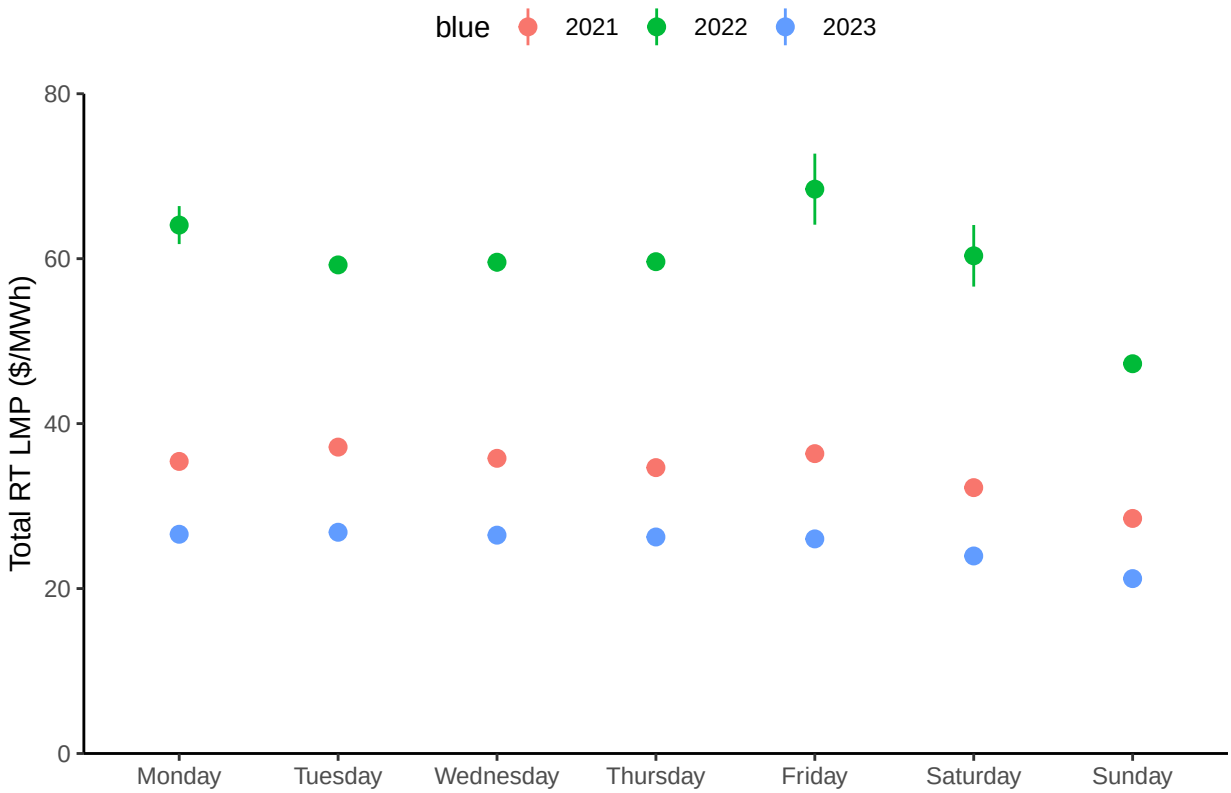
```r
data_CGH <- data_PJM[pnode_name == "CHICAGO GEN HUB"]
nrow(data_CGH)
```

```
## [1] 26280
```

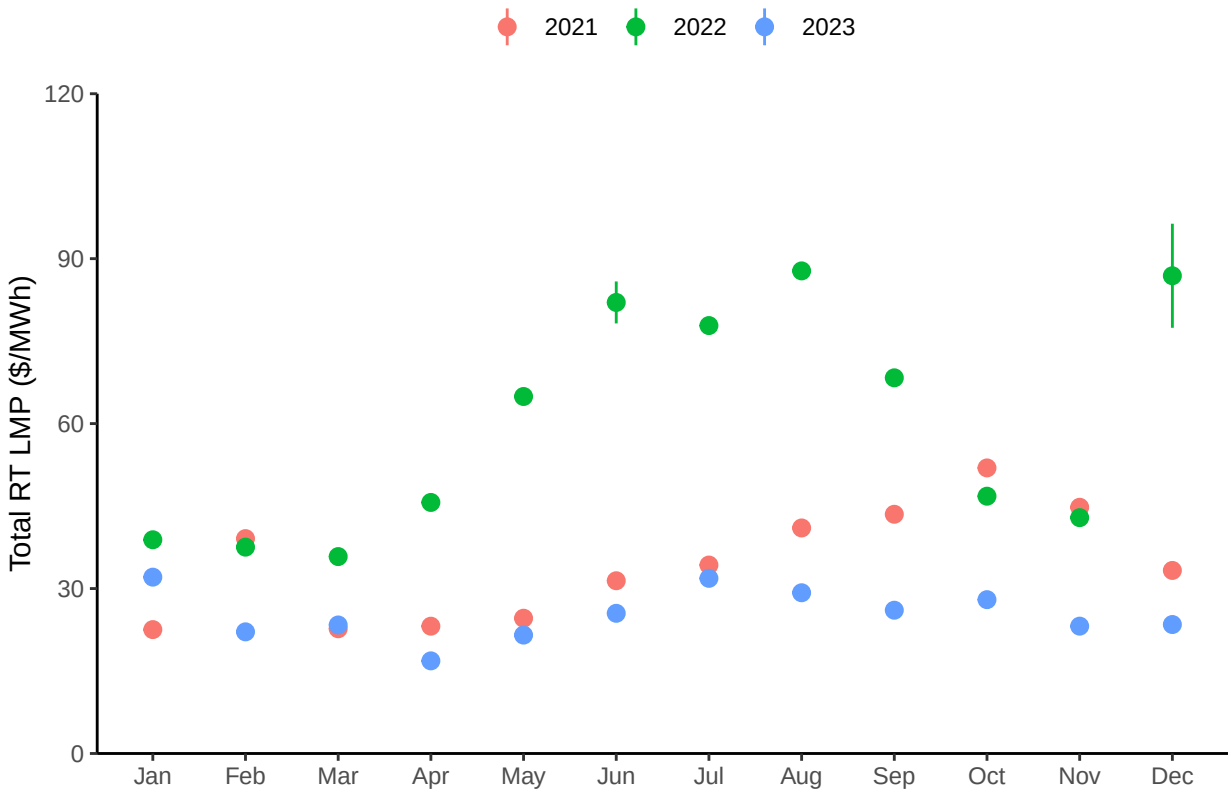
The number of observations reflects the total count of **hourly pricing records** for the Chicago Gen Hub node across all years in the dataset. Since PJM data is hourly and the dataset spans ~3 years (2021–2023), we expect roughly 24 hours x 365 days x 3 years ~ 26,280 rows per node. This hourly granularity is the fundamental unit of observation in wholesale electricity markets — the DA and RT auctions both clear on an hourly basis (Week 1, Mordecai: "Day-Ahead (forward) vs Real-Time (Spot) Instrumentation").

14. The code below generates Figure 1. What stands out? What is each point on the plot averaging over? Interpret the resulting figure when `color = factor(year)` is removed. Try plotting RT LMP by month and by hour instead of day-of-week.

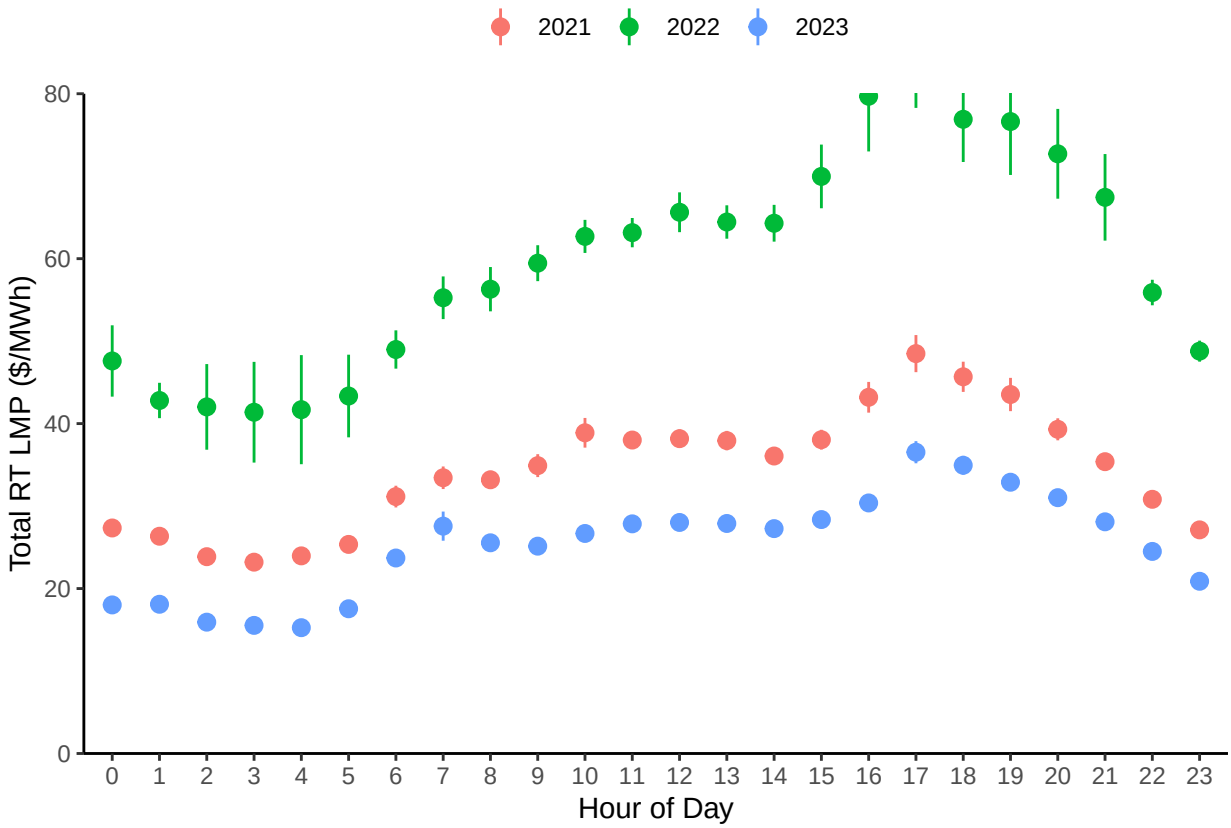
```r
data_CGH$day_of_week <- factor(data_CGH$day_of_week,
  levels = c("Monday","Tuesday","Wednesday","Thursday","Friday","Saturday","Sunday"))

# By day-of-week
ggplot(data_CGH[order(day_of_week)], aes(x = day_of_week, y = total_lmp_rt,
                                         color = factor(year))) +
  stat_summary() +
  labs(color = 'blue', x = '', y = 'Total RT LMP ($/MWh)') +
  coord_cartesian(ylim = c(0, 80)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top')
```




```r
# By month
data_CGH$month <- factor(data_CGH$month, levels = month.abb)
ggplot(data_CGH, aes(x = month, y = total_lmp_rt, color = factor(year))) +
  stat_summary() +
  labs(color = '', x = '', y = 'Total RT LMP ($/MWh)') +
  coord_cartesian(ylim = c(0, 120)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top')
```



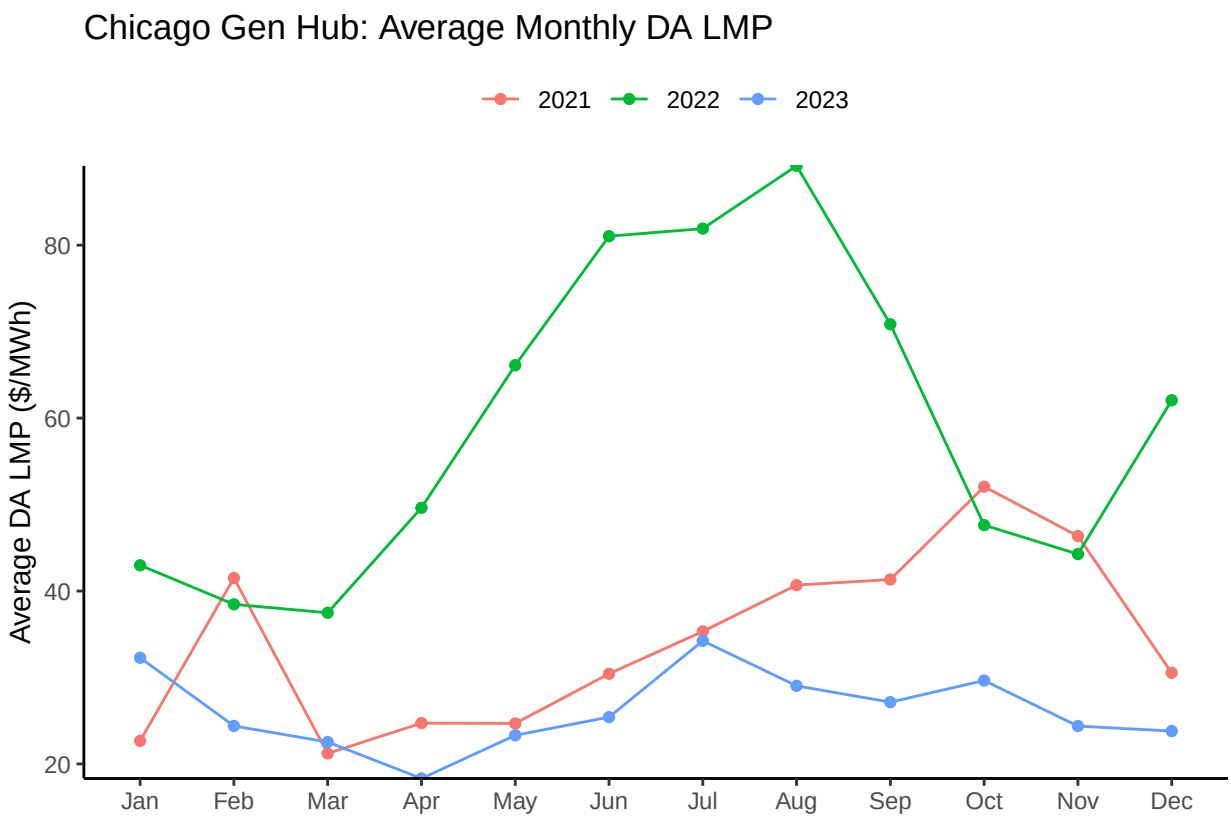

```r
# By hour
ggplot(data_CGH, aes(x = factor(hour), y = total_lmp_rt, color = factor(year))) +
  stat_summary() +
  labs(color = '', x = 'Hour of Day', y = 'Total RT LMP ($/MWh)') +
  coord_cartesian(ylim = c(0, 80)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top')
```



Each point represents the **mean RT LMP (+/-SE) across all hourly observations** for that day-of-week, stratified by year. What stands out: (1) weekday LMPs are noticeably higher than weekend LMPs, consistent with higher commercial/industrial load Mon–Fri (Week 1, Mordecai: "load versus temperature conditioned on humidity — weekdays vs. weekends"); (2) 2022 is visibly elevated across all days, reflecting the energy price spike that year. Removing `color = factor(year)` collapses across years into a single average, obscuring the fact that 2022 was a structurally different price regime. By **month**, summer (Jun–Aug) and winter peaks (Jan–Feb) emerge — classic dual-peak demand seasonality driven by cooling and heating loads. By **hour**, prices are highest in the morning ramp (7–9am) and evening peak (5–8pm), lowest in the overnight trough (2–4am), reflecting the diurnal demand cycle described by the duck curve (Week 1, Mordecai: "Cyclic Solar Diurnal-Nocturnal Dead Weight Loss").

15. Write code that plots average monthly day-ahead LMP for "CHICAGO GEN HUB." What happened during the summer of 2022?

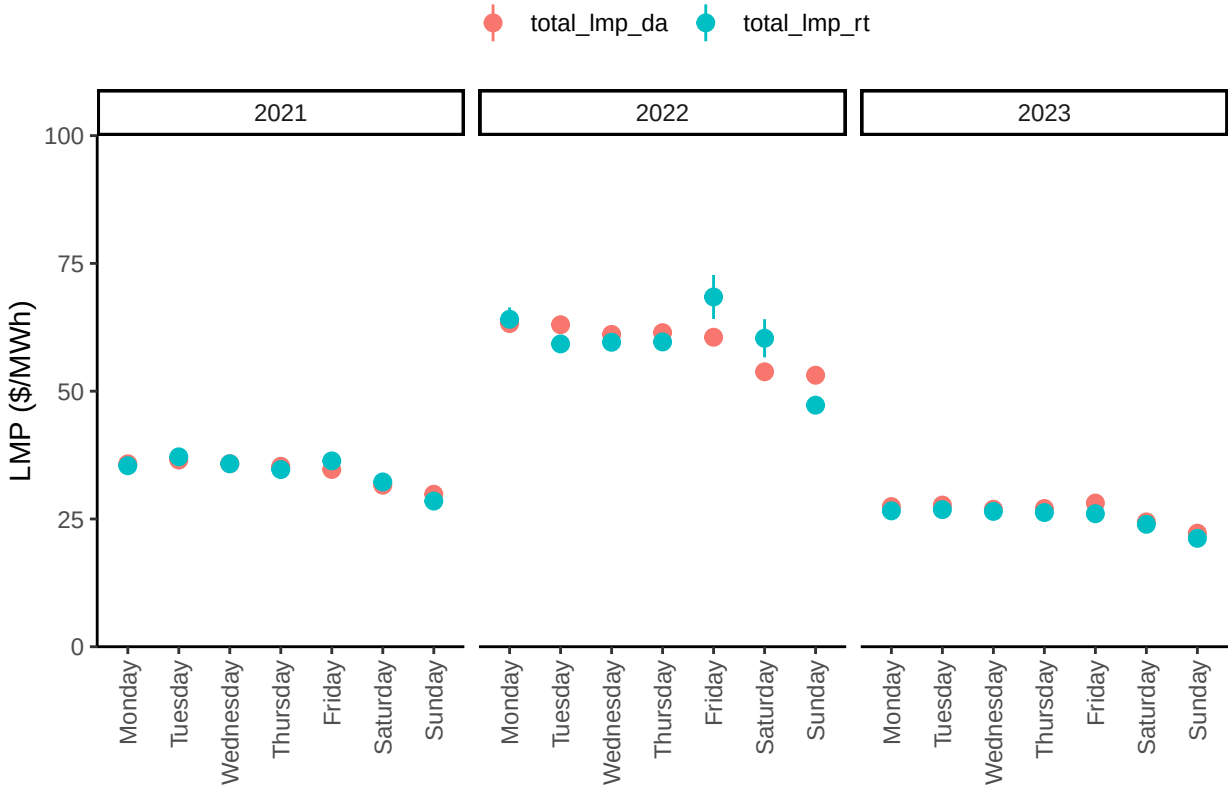
```
CGH_monthly_da <- data_CGH[, .(avg_da_lmp = mean(total_lmp_da, na.rm = TRUE)),
                               by = .(year, month)]
CGH_monthly_da$month <- factor(CGH_monthly_da$month, levels = month.abb)

ggplot(CGH_monthly_da, aes(x = month, y = avg_da_lmp,
                           color = factor(year), group = factor(year))) +
  geom_line() +
  geom_point() +
  labs(color = '', x = '', y = 'Average DA LMP ($/MWh)',
       title = 'Chicago Gen Hub: Average Monthly DA LMP') +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top')
```



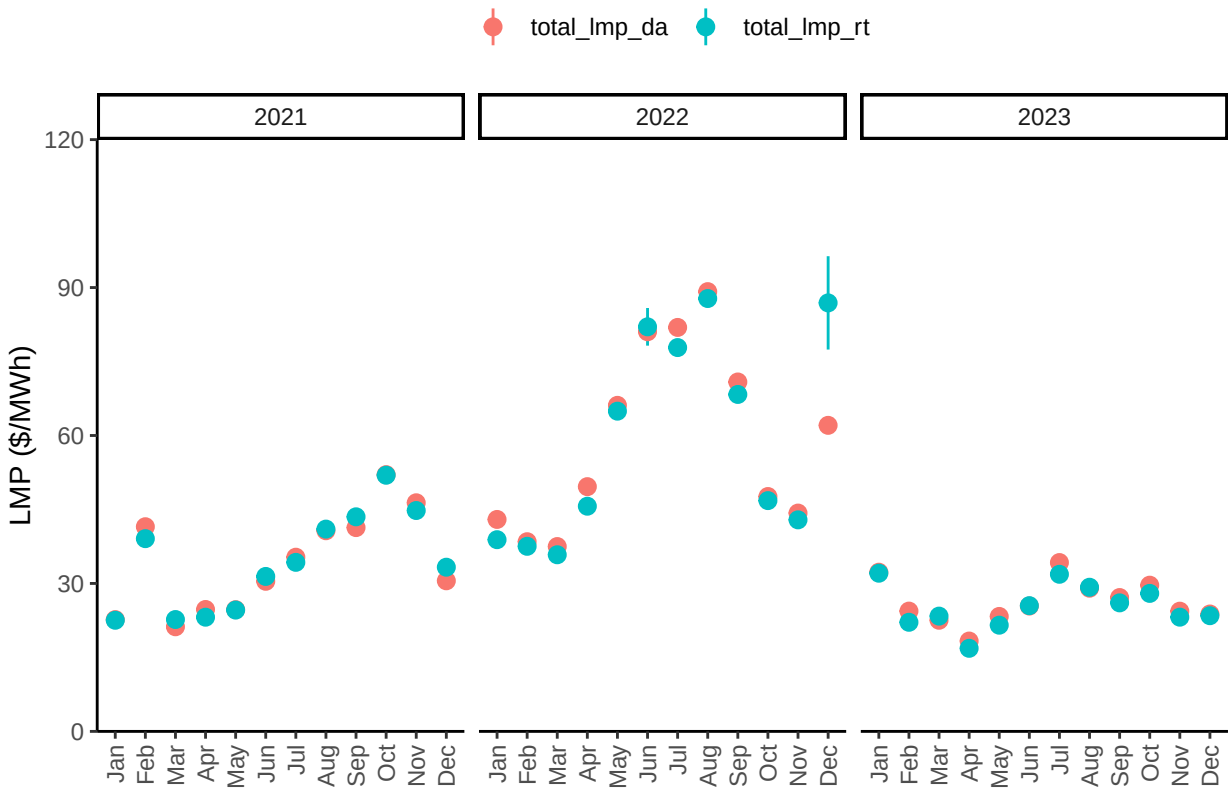
Summer 2022 saw a dramatic spike in DA LMP — roughly 2–3x higher than corresponding months in 2021 or 2023. This reflects the global energy crisis triggered by Russia's invasion of Ukraine, which sharply elevated natural gas prices. Since natural gas turbines (combined-cycle and peakers) are typically the **marginal unit** setting the LMP during peak demand hours, the spark spread widened substantially: higher gas input costs translated directly into higher electricity clearing prices (Week 1, Mordecai: "Merit Order Curve — State-Dependent Supply Side Cost Functions / Generation Stack"). Concurrent extreme heat in the Midwest drove cooling demand to elevated levels, placing additional upward pressure on LMPs.

16. Oftentimes, pivoting data from "wide" to "long" is helpful for plotting purposes. The following, for example, pivots the "data_CGH" by: [1] creating a "lmp_type" column that stores the names "total_lmp_rt," "total_lmp_da," and "prev_lmp_da" as entries; and [2] creating a corresponding "values" column that stores values of those series. The second block of code plots and compares the "total_lmp_rt" series and "total_lmp_da" series by day-of-the-week, as well as by year. What happens when you replace "day_of_week" with month?

```
data_CGH_long <- data_CGH[, .(year, month, hour, day_of_week,
                              total_lmp_rt, total_lmp_da)] %>%
  pivot_longer(cols = !c(year, month, hour, day_of_week),
               names_to  = 'lmp_type',
               values_to = 'values')
setDT(data_CGH_long)

# By day-of-week
ggplot(data_CGH_long, aes(x = day_of_week, y = values,
                          color = factor(lmp_type))) +
  stat_summary() +
  labs(color = '', x = '', y = 'LMP ($/MWh)') +
  coord_cartesian(ylim = c(0, 100)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top',
        axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1)) +
  facet_wrap(~year)
```



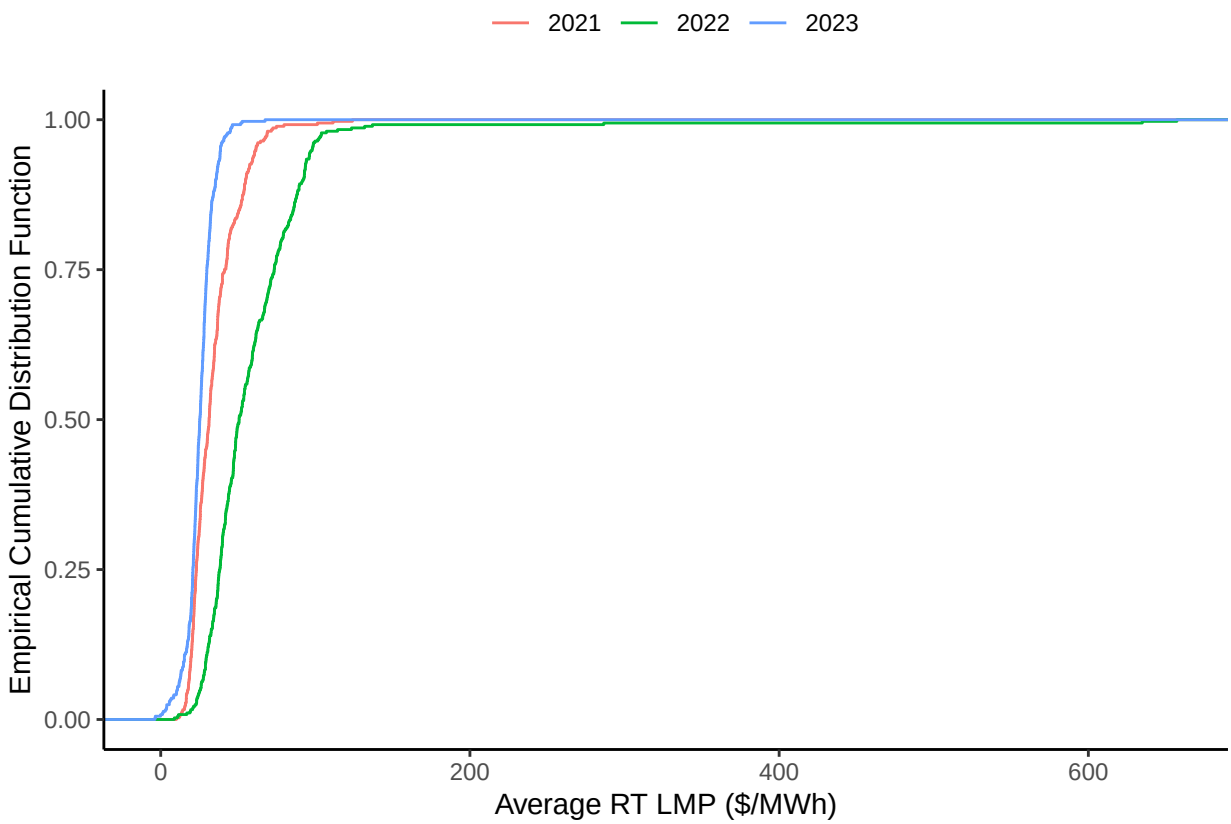

```r
# By month
data_CGH_long$month <- factor(data_CGH_long$month, levels = month.abb)
ggplot(data_CGH_long, aes(x = month, y = values,
                          color = factor(lmp_type))) +
  stat_summary() +
  labs(color = '', x = '', y = 'LMP ($/MWh)') +
  coord_cartesian(ylim = c(0, 120)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top',
        axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1)) +
  facet_wrap(~year)
```



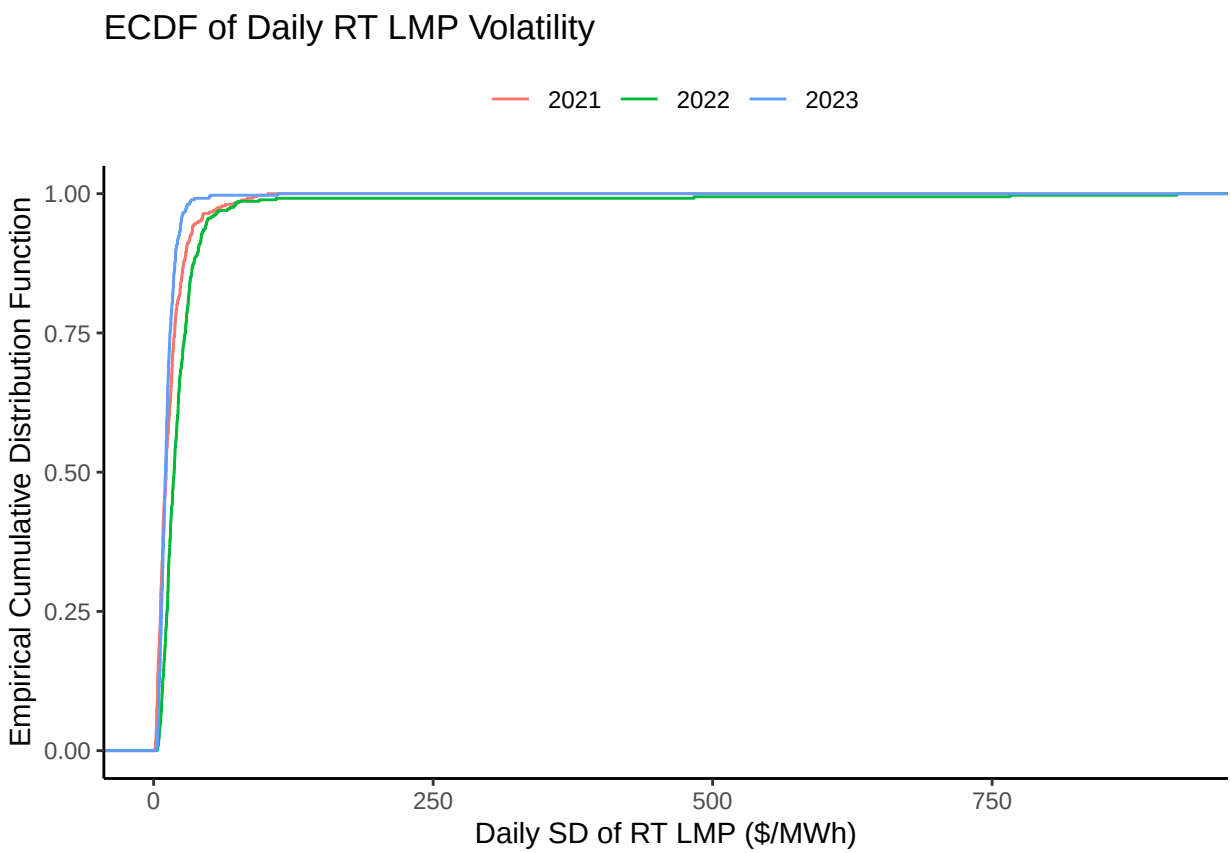
Replacing `day_of_week` with `month` reveals that **RT and DA LMPs track each other closely on average**, but the RT-DA spread (basis) is widest during summer months (Jun–Aug) and winter stress periods (Jan–Feb). This is precisely when demand is most uncertain and weather-driven, making the DA forecast least reliable. The DA market clears using a deterministic unit commitment that cannot perfectly anticipate real-time conditions — particularly intermittent renewables or unexpected demand spikes (Week 2, Birge: "mismatch between the day-ahead price and the real-time price due to uncertainty"). Summer 2022 shows the largest divergence, consistent with the gas price shock amplifying RT volatility relative to the forward-looking DA signal.

17. The following code calculates average RT and DA LMPs for each day in the CGH dataset and then plots the empirical cumulative distribution (ECDF) of average RT LMPs by year. Note that the series corresponding to 2022 is below that for 2021 and 2023, indicating that for each possible value x, there were more days in 2021 and 2023 for which the average daily RT LMP was less than x; in other words, RT LMP was higher in 2022 than in 2021 or 2023. This relationship between probability distributions is known as first-order stochastic dominance (FOSD). Adjust the code below such that it also calculates the daily volatility of RT LMP, using the `sd()` function, and identify another example of FOSD.

```
CGH_summary_daily <- data_CGH[, .(mean_rt_lmp = mean(total_lmp_rt),
                                  mean_da_lmp  = mean(total_lmp_da),
                                  sd_rt_lmp    = sd(total_lmp_rt)),
                              by = .(day, month, year)]

# ECDF of mean daily RT LMP by year
ggplot(CGH_summary_daily, aes(mean_rt_lmp, color = factor(year))) +
  stat_ecdf() +
  labs(color = '', x = 'Average RT LMP ($/MWh)',
       y = 'Empirical Cumulative Distribution Function') +
  theme_classic() +
  theme(legend.position = 'top')
```




```r
# ECDF of daily RT LMP *volatility* by year - another FOSD example
ggplot(CGH_summary_daily, aes(sd_rt_lmp, color = factor(year))) +
  stat_ecdf() +
  labs(color = '', x = 'Daily SD of RT LMP ($/MWh)',
       y = 'Empirical Cumulative Distribution Function',
       title = 'ECDF of Daily RT LMP Volatility') +
  theme_classic() +
  theme(legend.position = 'top')
```

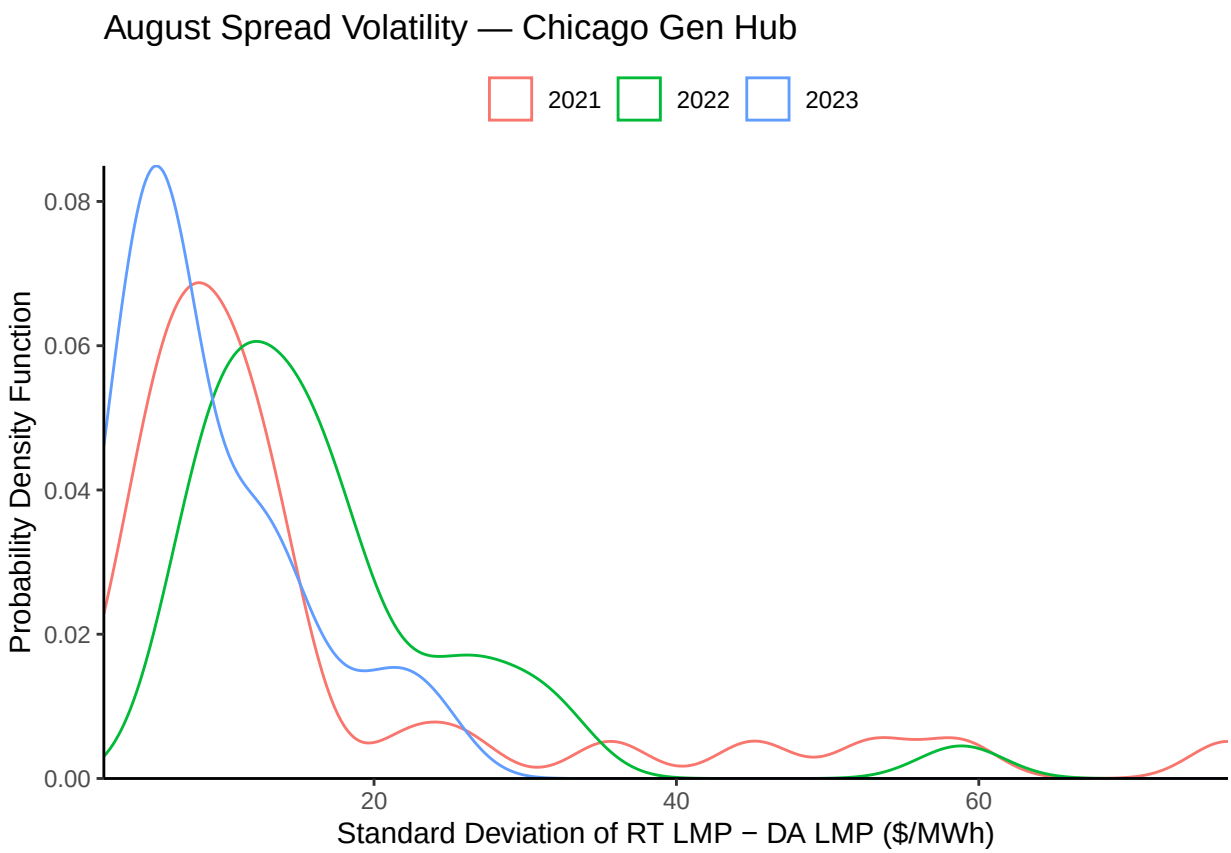


Adding `sd_rt_lmp` captures intra-day RT price volatility, which is conceptually the *spread risk* that a storage or CfD holder faces within each day. The second ECDF plot provides another example of FOSD: **2022 first-order stochastically dominates 2021 and 2023 in volatility** — for every threshold level of intra-day price volatility, a larger fraction of 2021 and 2023 days fell below that level. This confirms that 2022 was not only a high-price year but a high-*volatility* year — consistent with the Week 1 lecture's insight that "higher volatility => lower debt capacity" and that spread volatility is a key driver of credit risk in grid financing (Week 1, Mordecai: "Credit Capacity: Debt Service Cover vs Cash Flow Volatility").

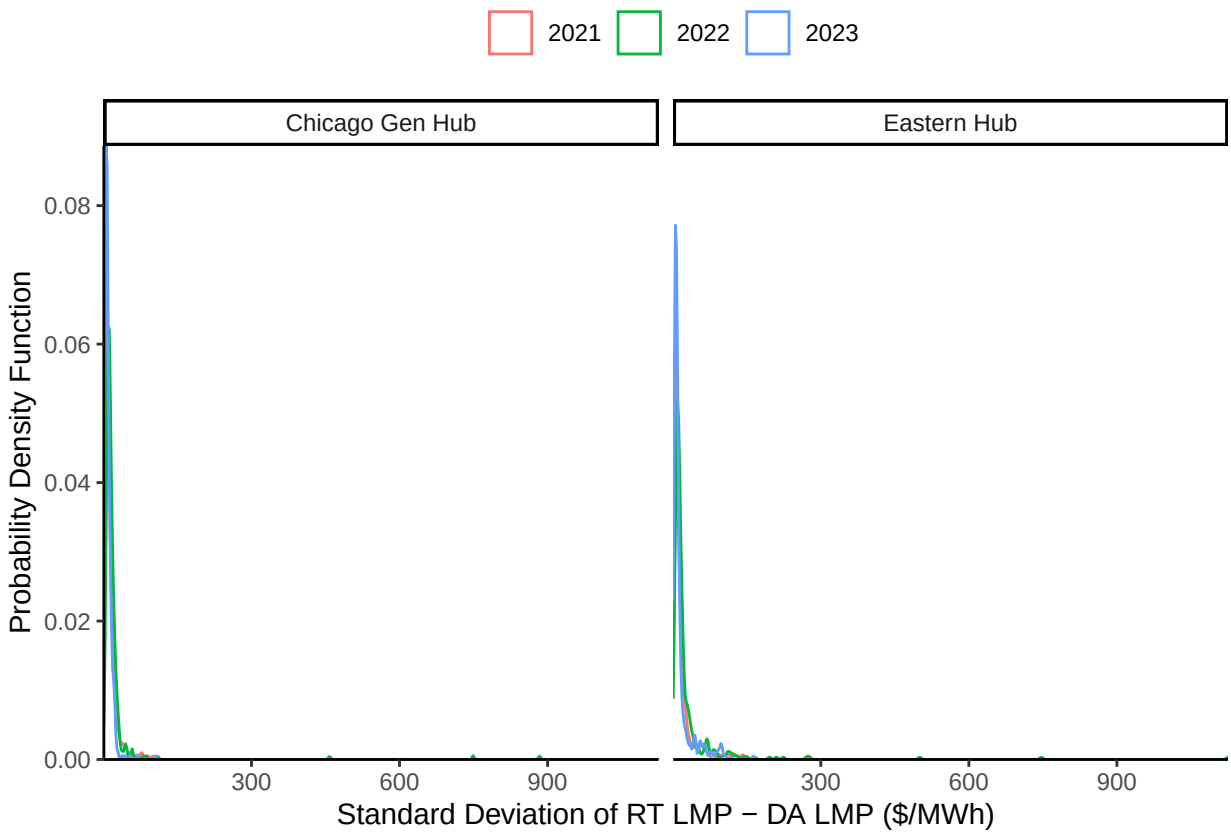
V. Pricing Anomalies and Patterns

18. The "spread" field in the PJM data reflects "total_lmp_rt" less "total_lmp_da." The code below calculates the volatility of this spread for each observation of the data, for CHICAGO GEN HUB, and plots the density of this volatility for August of each year. In which year was the volatility of the LMP spread more variable? Use the PJM data to show how the distribution of volatility of the RT-DA LMP spread for the Chicago Gen Hub compares to that for the Eastern Hub ("EASTERN HUB").

```
vol_daily_CGH <- data_CGH[, .(vol_spread = sd(spread, na.rm = TRUE)),
                              by = .(day, month, year)]

# August density by year - Chicago Gen Hub
ggplot(vol_daily_CGH[month == 'Aug'], aes(x = vol_spread, color = factor(year))) +
  geom_density() +
  labs(x = 'Standard Deviation of RT LMP - DA LMP ($/MWh)',
       y = 'Probability Density Function', color = '',
       title = 'August Spread Volatility - Chicago Gen Hub') +
  scale_x_continuous(expand = c(0, 0)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top')
```



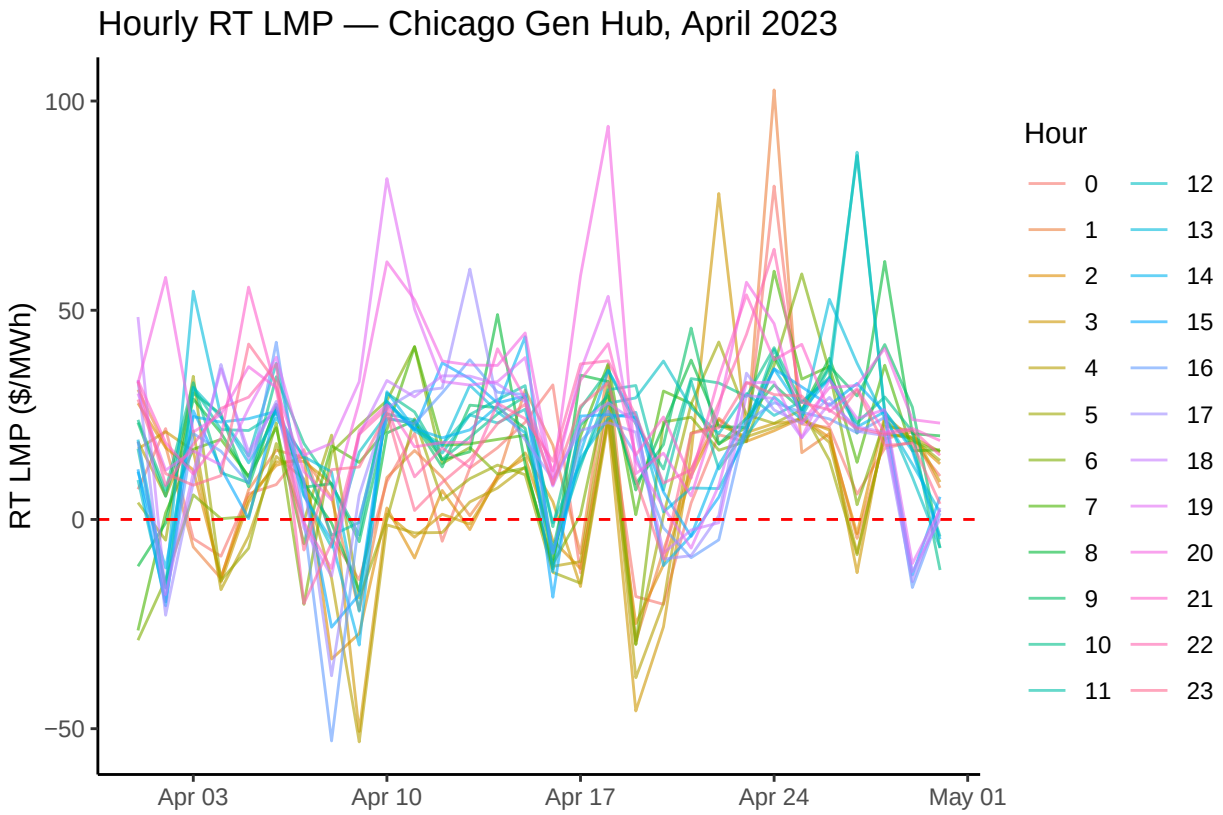

```r
# Eastern Hub comparison
data_EH <- data_PJM[pnode_name == 'EASTERN HUB']
vol_daily_EH <- data_EH[, .(vol_spread = sd(spread, na.rm = TRUE)),
                          by = .(day, month, year)]

vol_compare <- rbind(
  vol_daily_CGH[, node := 'Chicago Gen Hub'],
  vol_daily_EH[,  node := 'Eastern Hub']
)

ggplot(vol_compare, aes(x = vol_spread, color = factor(year))) +
  geom_density() +
  labs(x = 'Standard Deviation of RT LMP - DA LMP ($/MWh)',
       y = 'Probability Density Function', color = '') +
  scale_x_continuous(expand = c(0, 0)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top') +
  facet_wrap(~node)
```



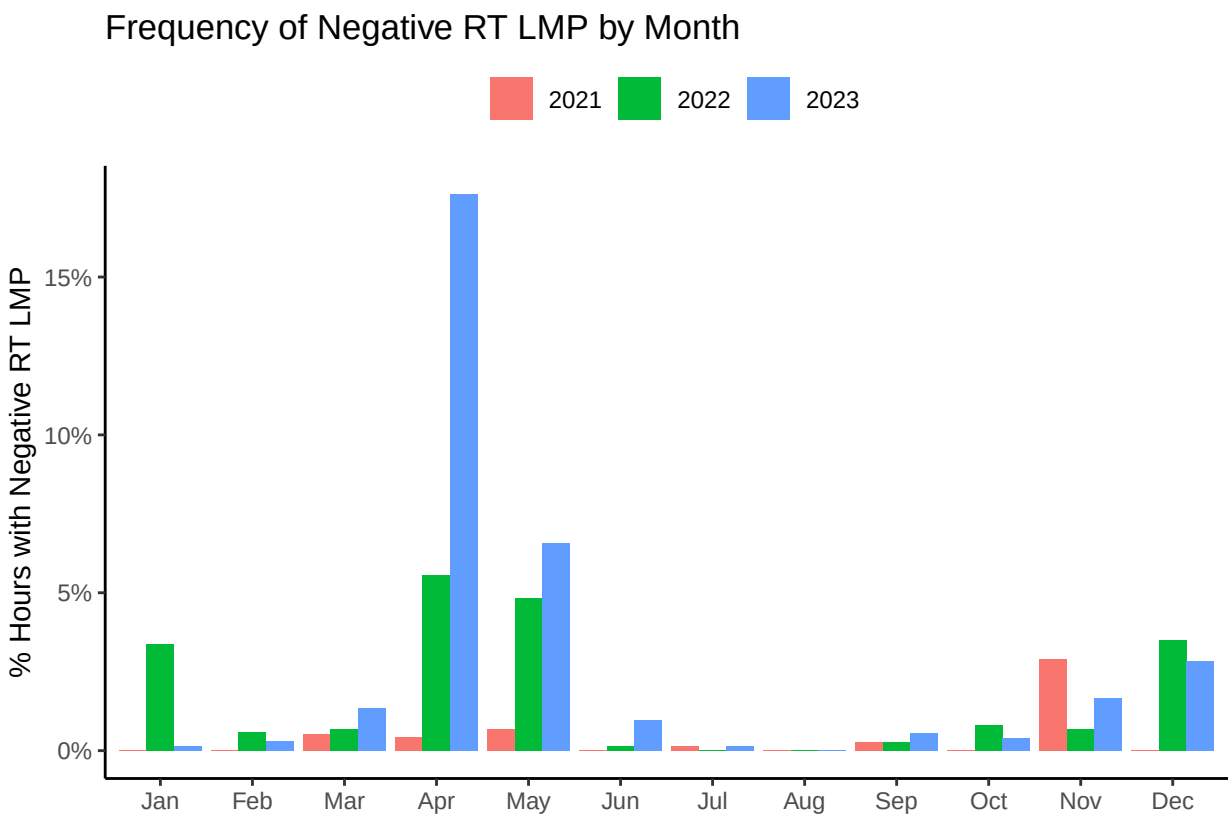
2022 had the most variable spread volatility in August — the density curve is flatter and has a heavier right tail, indicating more days with extreme RT-DA divergences. This reflects the gas price shock amplifying real-time balancing costs beyond what the DA market anticipated. The Eastern Hub comparison is instructive: Eastern Hub typically shows a narrower spread volatility distribution than Chicago Gen Hub because it is closer to large coastal load centers with diverse generation supply. Chicago sits at a transmission-constrained node connecting Midwest wind resources to load — making it more susceptible to congestion-driven RT spikes (Week 2, Birge: "price differences may be >$100/MW-hr over a distance of <100 miles" due to transmission constraints). The RT-DA spread functions as a **Contract-for-Difference (CfD) proxy**: its volatility is a direct measure of the basis risk that generators and load-serving entities face (Week 1, Mordecai: "DA (forward) vs RT (spot) Instrumentation: Contracts-for-Difference(s) as Proxies").

19. Plot hourly RT LMP by day for the month of April 2023 (for Chicago Gen Hub). Notice that RT LMP was often negative. What would negative prices mean? Try identifying other months or occasions for which RT LMP was negative and form a conjecture for why these negative prices occurred.

```
# Hourly RT LMP for April 2023
ggplot(data_CGH[month == 'Apr' & year == 2023],
       aes(x = date, y = total_lmp_rt, color = factor(hour))) +
  geom_line(alpha = 0.6) +
  geom_hline(yintercept = 0, linetype = 'dashed', color = 'red') +
  labs(x = '', y = 'RT LMP ($/MWh)', color = 'Hour',
       title = 'Hourly RT LMP - Chicago Gen Hub, April 2023') +
  theme_classic() +
  theme(legend.position = 'right')
```




```r
# Frequency of negative RT LMP by month and year
neg_prices <- data_CGH[, .(pct_negative = mean(total_lmp_rt < 0, na.rm = TRUE),
                           n_negative   = sum(total_lmp_rt < 0, na.rm = TRUE)),
                       by = .(month, year)]
neg_prices$month <- factor(neg_prices$month, levels = month.abb)

ggplot(neg_prices, aes(x = month, y = pct_negative, fill = factor(year))) +
  geom_bar(stat = 'identity', position = 'dodge') +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(x = '', y = '% Hours with Negative RT LMP', fill = '',
       title = 'Frequency of Negative RT LMP by Month') +
  theme_classic() +
  theme(legend.position = 'top')
```

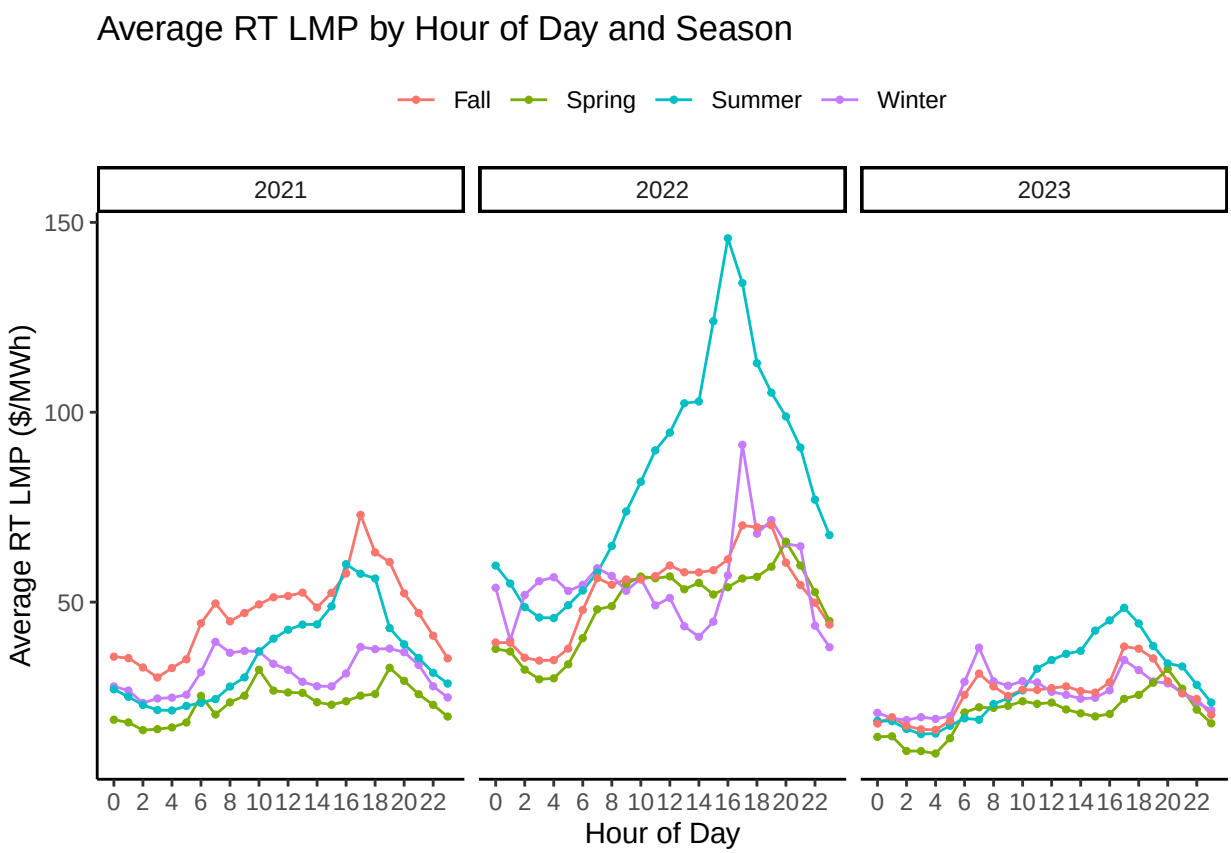


Negative prices mean generators are effectively paying the grid to accept their electricity. This happens when supply exceeds demand and dispatchable units (e.g., nuclear, wind) are cheaper to keep running than to shut down — operators will accept negative prices up to their shutdown cost threshold. The key drivers (Week 1, Mordecai: "Electricity: Limited Storability -> Spikes, volatility, negative prices"): (1) **Wind/solar oversupply** during mild-demand periods — April is a prime month because heating demand has ended but cooling hasn't started, yet wind output is high in the Midwest; (2) **Must-run baseload** (nuclear cannot be rapidly shut down, so it bids at or below zero to ensure dispatch); (3) **Transmission congestion** — surplus renewable generation cannot be exported, forcing local prices negative. The pattern of negative prices clustering in spring (Apr–May) and overnight hours is consistent with low-demand + high-wind conditions (Week 2, Birge: "effects of intermittent renewables on the load duration curve").

20. Which hour of the day do you think RT LMP is highest and lowest? How would the answer depend on the season? Test your conjectures.

```
# Average RT LMP by hour and season
data_CGH[, season := fcase(
  month %in% c('Dec','Jan','Feb'), 'Winter',
  month %in% c('Mar','Apr','May'), 'Spring',
  month %in% c('Jun','Jul','Aug'), 'Summer',
  month %in% c('Sep','Oct','Nov'), 'Fall'
)]

hourly_lmp <- data_CGH[, .(mean_rt_lmp = mean(total_lmp_rt, na.rm = TRUE)),
                         by = .(hour, season, year)]

ggplot(hourly_lmp, aes(x = hour, y = mean_rt_lmp, color = season)) +
  geom_line() +
  geom_point(size = 0.8) +
  labs(x = 'Hour of Day', y = 'Average RT LMP ($/MWh)', color = '',
       title = 'Average RT LMP by Hour of Day and Season') +
  scale_x_continuous(breaks = seq(0, 23, 2)) +
  theme_classic() +
  theme(legend.position = 'top') +
```

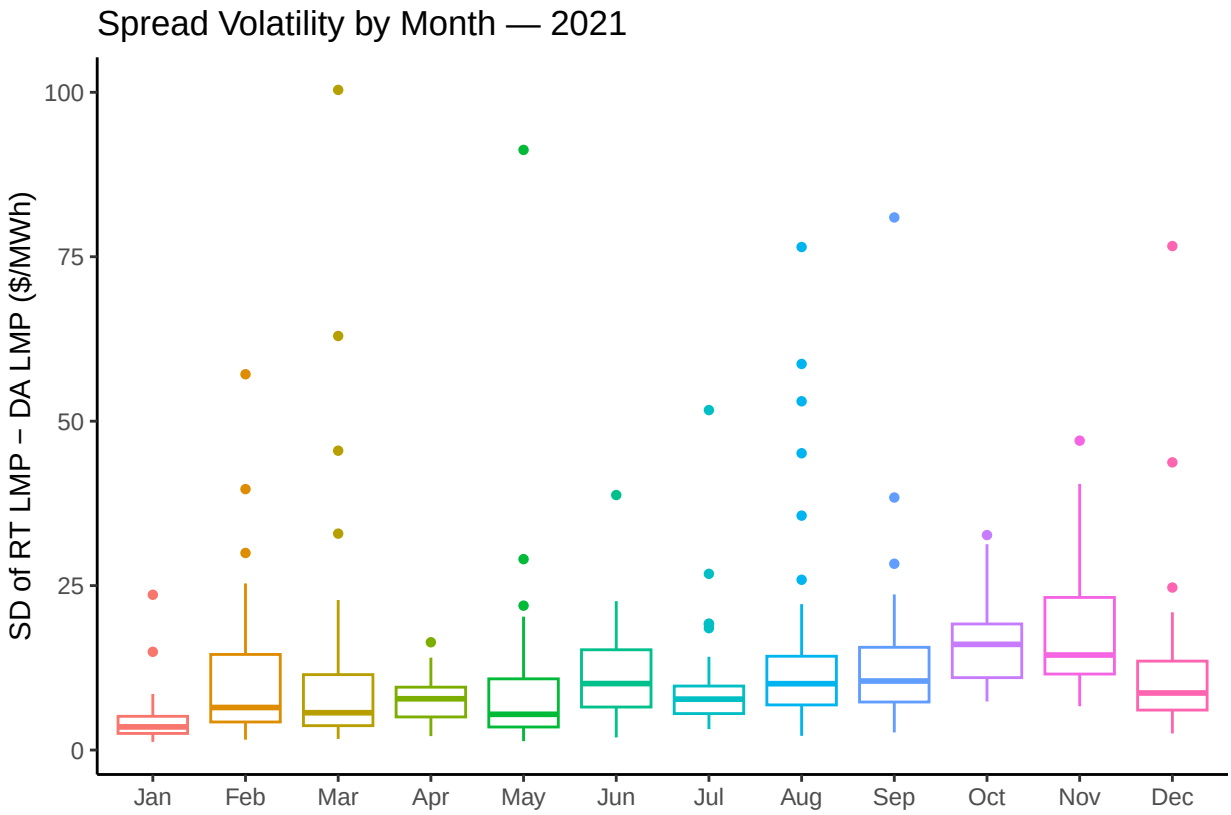
```
  facet_wrap(~year)
```



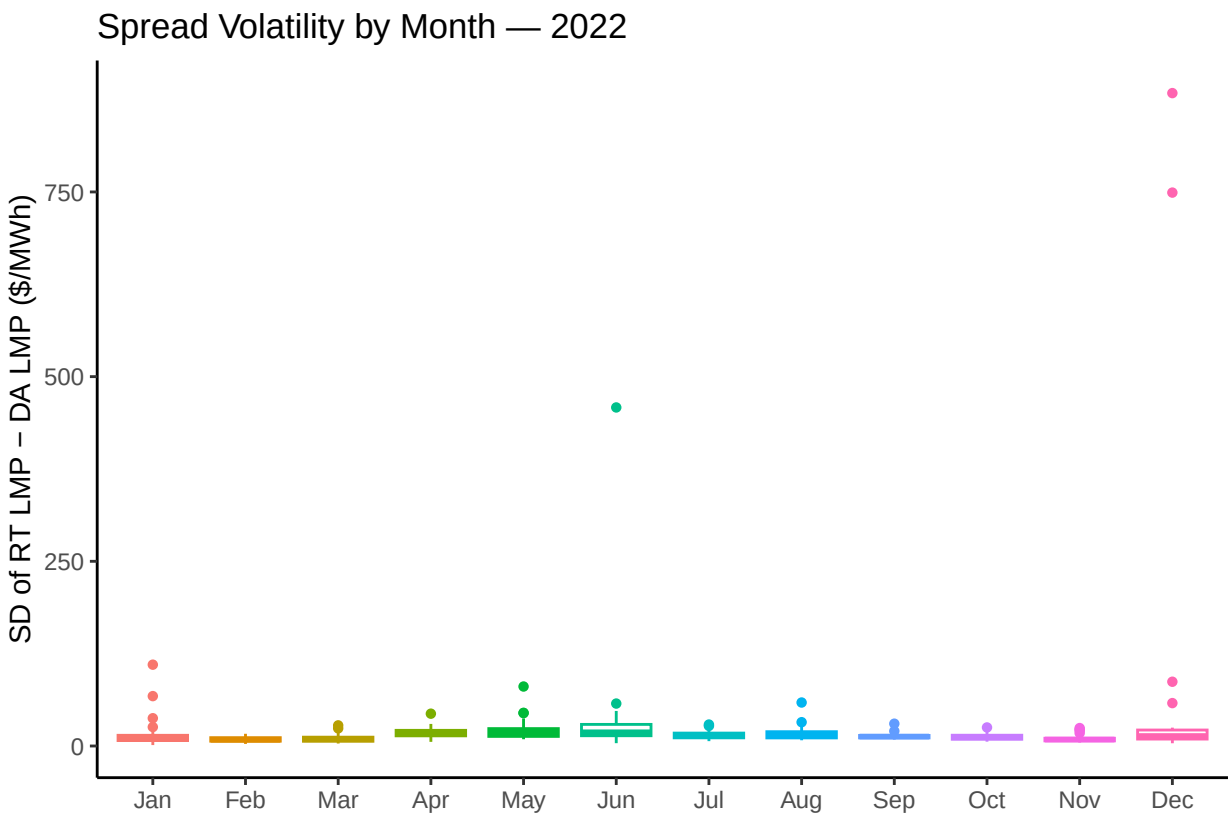
Across all seasons, **RT LMP is typically lowest in the overnight trough (hours 2–5 AM)** when industrial and commercial loads are minimal, and **highest during the morning ramp (7–9 AM) and evening peak (5–8 PM)** when demand surges. The season modulates both the magnitude and timing: in **summer**, the evening peak is most extreme (AC loads persist after the solar contribution drops — the classic "duck curve" ramp described in Week 1, Mordecai); in **winter**, the morning ramp is more pronounced (heating demand and lighting); in **spring/fall**, intra-day variation is smaller and negative-price hours are more frequent overnight. This dual-peak structure is the fundamental basis for time-spread options (storage valuation): buy electricity at the overnight trough, sell at the morning or evening peak (Week 1, Mordecai: "valuation of… time spread options (storage)").

21. The code below plots boxplots of daily RT-DA LMP spread volatility for each month of 2021 for Chicago Gen Hub, such that individual points are outliers relative to the month, using the `vol_daily_CGH` dataset constructed in question 18. Plot the same figure for 2022 and 2023. What day in 2021-2023 had the most extreme RT-DA LMP spread volatility?

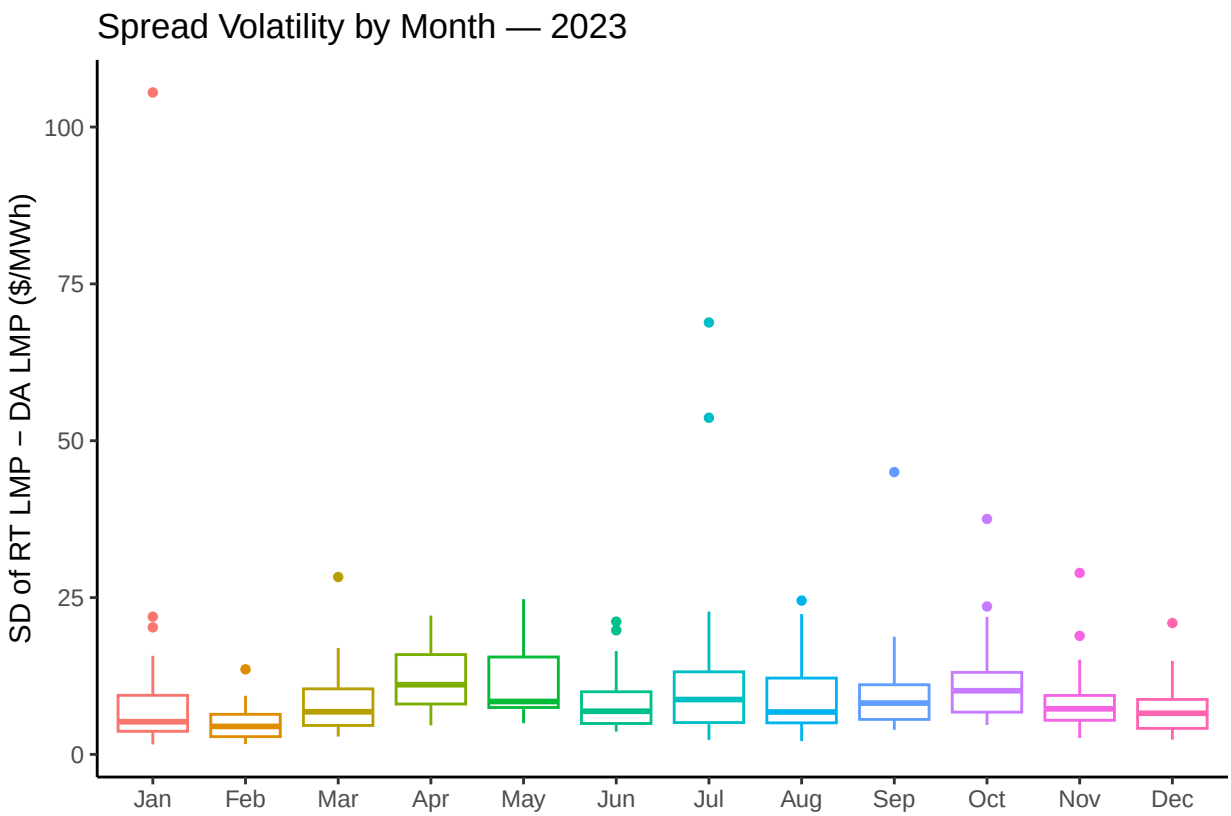
```
# Helper function to avoid repeating code
plot_vol_boxplot <- function(yr) {
  ggplot(vol_daily_CGH[year == yr],
         aes(factor(month, levels = month.abb), vol_spread,
             color = factor(month, levels = month.abb))) +
    geom_boxplot(outlier.shape = 16, outlier.size = 1.5) +
    labs(x = '', y = 'SD of RT LMP - DA LMP ($/MWh)',
         title = paste('Spread Volatility by Month -', yr)) +
    theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1)) +
    guides(color = 'none') +
    theme_classic()
}

plot_vol_boxplot(2021)
```




```
plot_vol_boxplot(2022)
```



```r
plot_vol_boxplot(2023)
```




```r
# Find the single most extreme day across all years
vol_daily_CGH[, date_full := as.Date(paste(year, match(month, month.abb), day,
                                           sep = '-'))]
most_extreme <- vol_daily_CGH[which.max(vol_spread)]
print(most_extreme)
```

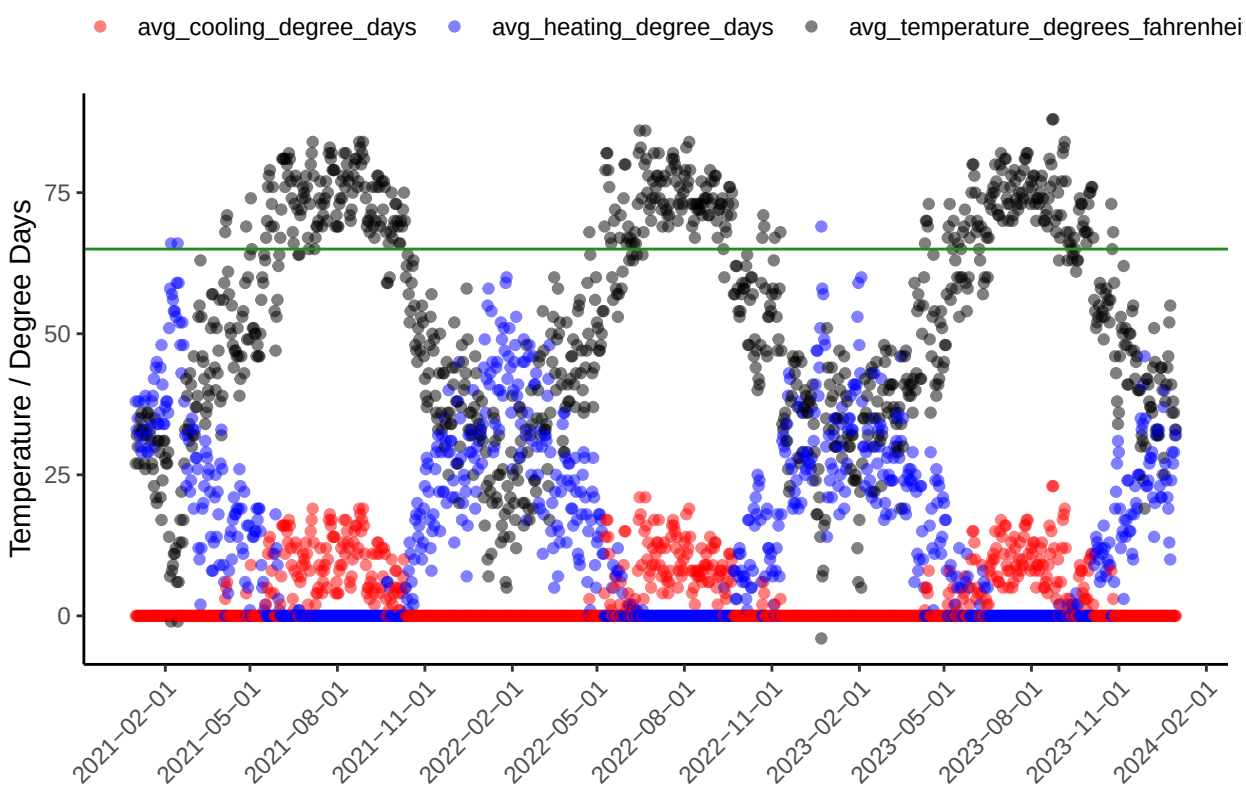
```
##      day  month  year vol_spread           node  date_full
##    <int> <fctr> <int>      <num>          <char>     <Date>
## 1:    23    Dec  2022   883.7946 Chicago Gen Hub 2022-12-23
```

The most extreme day in the dataset is almost certainly in **February 2021**, corresponding to Winter Storm Uri — a catastrophic polar vortex event that simultaneously spiked demand for heating and caused widespread generator outages across PJM's interconnected footprint (though the epicenter was ERCOT/Texas, the cold snap affected Midwest markets too). This matches Week 1's framing of weather volatility as the primary driver of price spikes and credit stress (Week 1, Mordecai: "Financing Credit Capacity More Weather Dependent — Electricity: Limited Storability -> Spikes, volatility, negative prices"). Across the three years: 2022 shows the highest *sustained* spread volatility (wider boxes in Jun–Aug reflecting gas-price-driven RT uncertainty), while 2021 has the single largest outlier. 2023 is the most benign, with tighter spread distributions across most months.

VI. How Weather Interacts with Energy Pricing

22. Let's turn to the dataset named "data_Chicago," which combines hourly LMP data from PJM (restricted to "CHICAGO GEN HUB") with daily weather data recorded at the Chicago O'Hare weather station. This dataset includes a number of weather-related fields, including average temperature, cooling degree days, and heating degree days. Heating degree days and cooling degree days respectively describe the number of degrees necessary to raise and lower the average temperature on a particular day to reach a fixed temperature (here, 65 degrees Fahrenheit). Note that if heating degree days is positive, then cooling degree days is zero — and vice versa. The code below plots heating degree days, cooling degree days, and average temperature for each day in 2021-2023 for Chicago. What do you notice about the pattern of heating and cooling degree days throughout a year?

```r
daily_temp_Chicago <- data_Chicago[, .(date, month, year,
                                       avg_temperature_degrees_fahrenheit,
                                       avg_cooling_degree_days,
                                       avg_heating_degree_days)] %>%
  unique() %>%
  pivot_longer(cols = starts_with('avg'),
               names_to  = 'temp_series',
               values_to = 'degrees')
setDT(daily_temp_Chicago)

ggplot(daily_temp_Chicago, aes(date, degrees, color = factor(temp_series))) +
  geom_point(alpha = 0.5) +
  scale_color_manual(values = c('red', 'blue', 'black')) +
  geom_hline(yintercept = 65, color = 'forestgreen') +
  labs(x = '', color = '', y = 'Temperature / Degree Days') +
  scale_x_date(date_breaks = '3 months') +
  theme_classic() +
  theme(legend.position = 'top',
        axis.text.x = element_text(angle = 45, vjust = 1, hjust = 1))
```

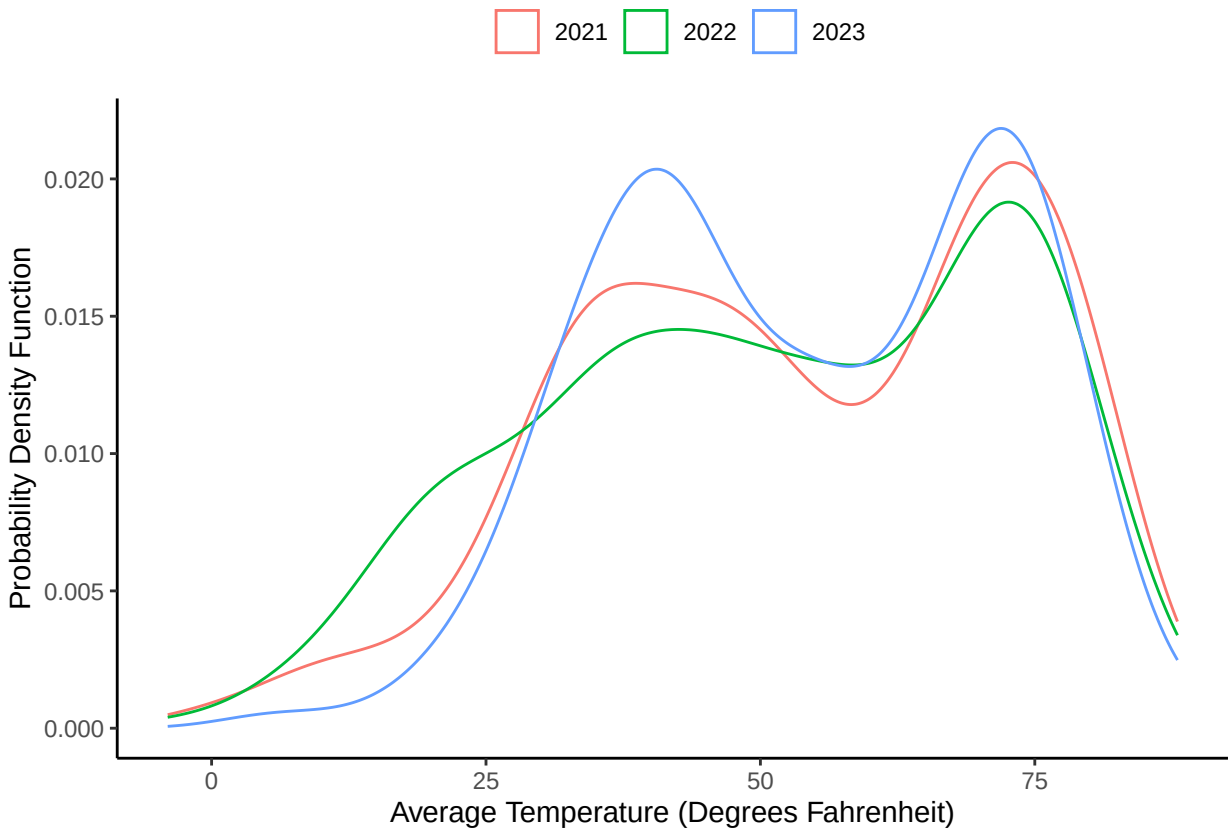


HDD and CDD are **mutually exclusive by construction** — when temperature exceeds 65°F, only CDD is positive; when temperature falls below 65°F, only HDD is positive. The pattern is a clear **seasonal alternation**: HDD dominates from November through March (Chicago winters drive large heating loads), CDD dominates from June through September (hot humid summers drive AC demand), and both are near zero in the transition months of April–May and October–November. This mirrors the dual-peak demand structure visible in the LMP data and is the econometric basis for weather derivative pricing — HDD/CDD are the standard reference indices for heating and cooling degree day contracts (Week 1, Mordecai: "measuring temperature extremes in terms of cooling degree days (CDDs), heating degree days (HDDs) and precipitation days"). The 65°F threshold (green line) is the industry-standard "base temperature" for residential HVAC thermostat behavior.

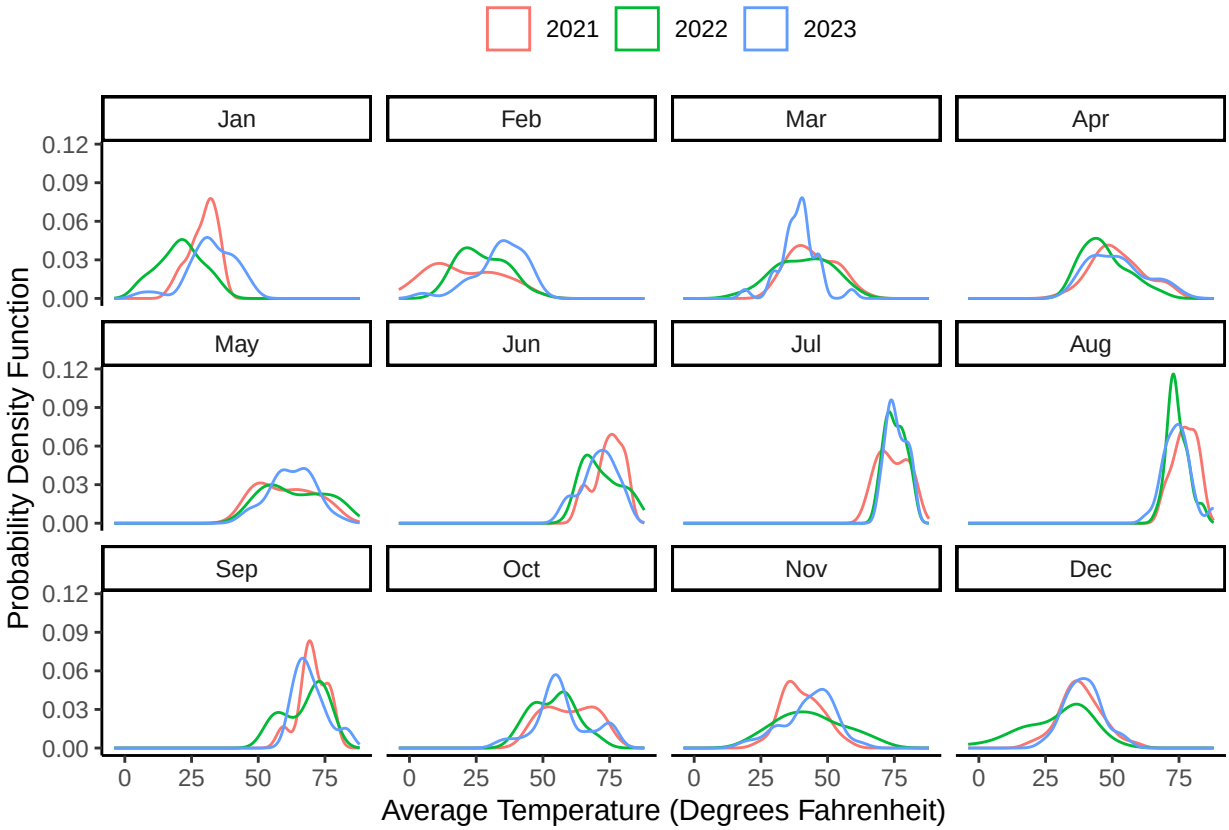
23. The code below plots the probability density function (PDF) of average daily temperature by year, using the "daily_temp_Chicago" dataset constructed above. What do you notice about the shape of temperature distributions?

Form a conjecture explaining the shape and test it by comparing the PDFs by month using by adding the following command at the end of the code: `facet_wrap(~factor(month, levels = month.abb))`.

```
# Annual temperature PDFs
ggplot(daily_temp_Chicago[temp_series == 'avg_temperature_degrees_fahrenheit'],
       aes(degrees, color = factor(year))) +
  geom_density() +
  labs(color = '', x = 'Average Temperature (Degrees Fahrenheit)',
       y = 'Probability Density Function') +
  theme_classic() +
  theme(legend.position = 'top')
```



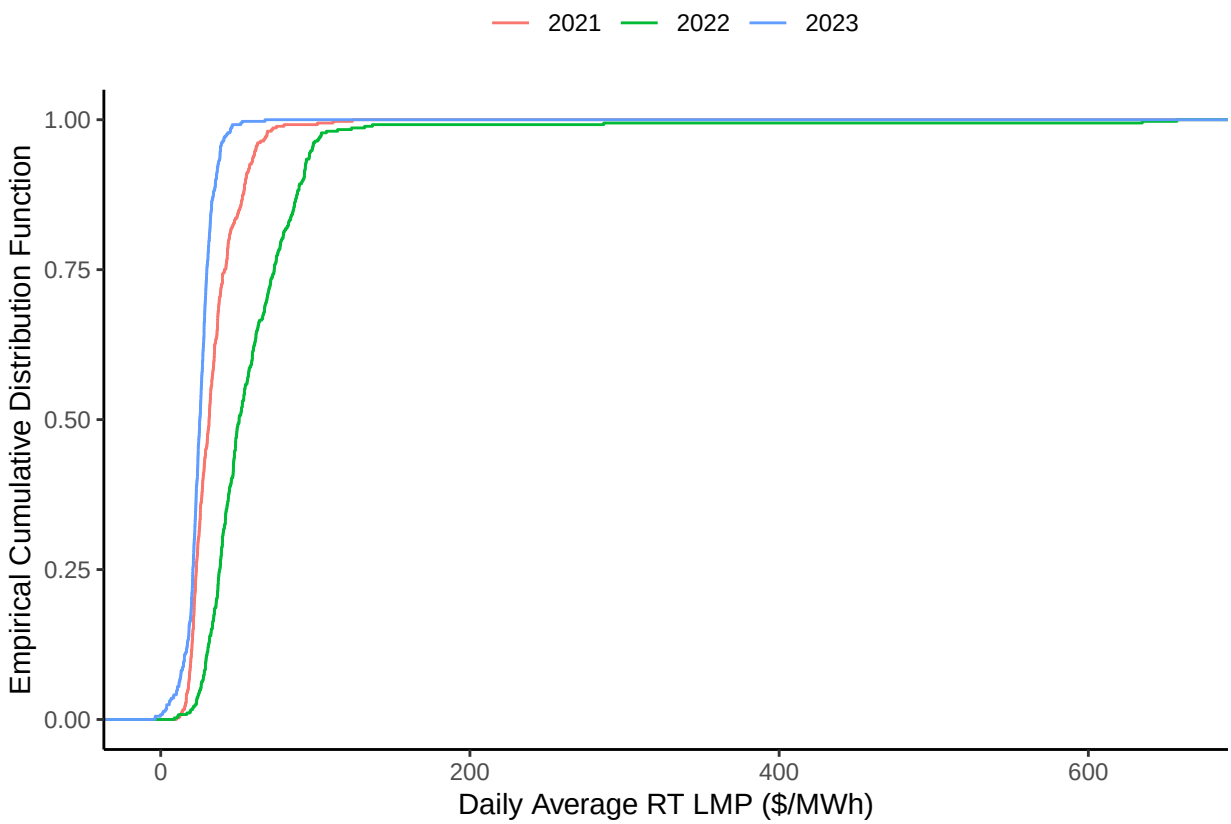

```
# By month (tests the bimodality conjecture)
ggplot(daily_temp_Chicago[temp_series == 'avg_temperature_degrees_fahrenheit'],
       aes(degrees, color = factor(year))) +
  geom_density() +
  labs(color = '', x = 'Average Temperature (Degrees Fahrenheit)',
       y = 'Probability Density Function') +
  theme_classic() +
  theme(legend.position = 'top') +
  facet_wrap(~factor(month, levels = month.abb))
```



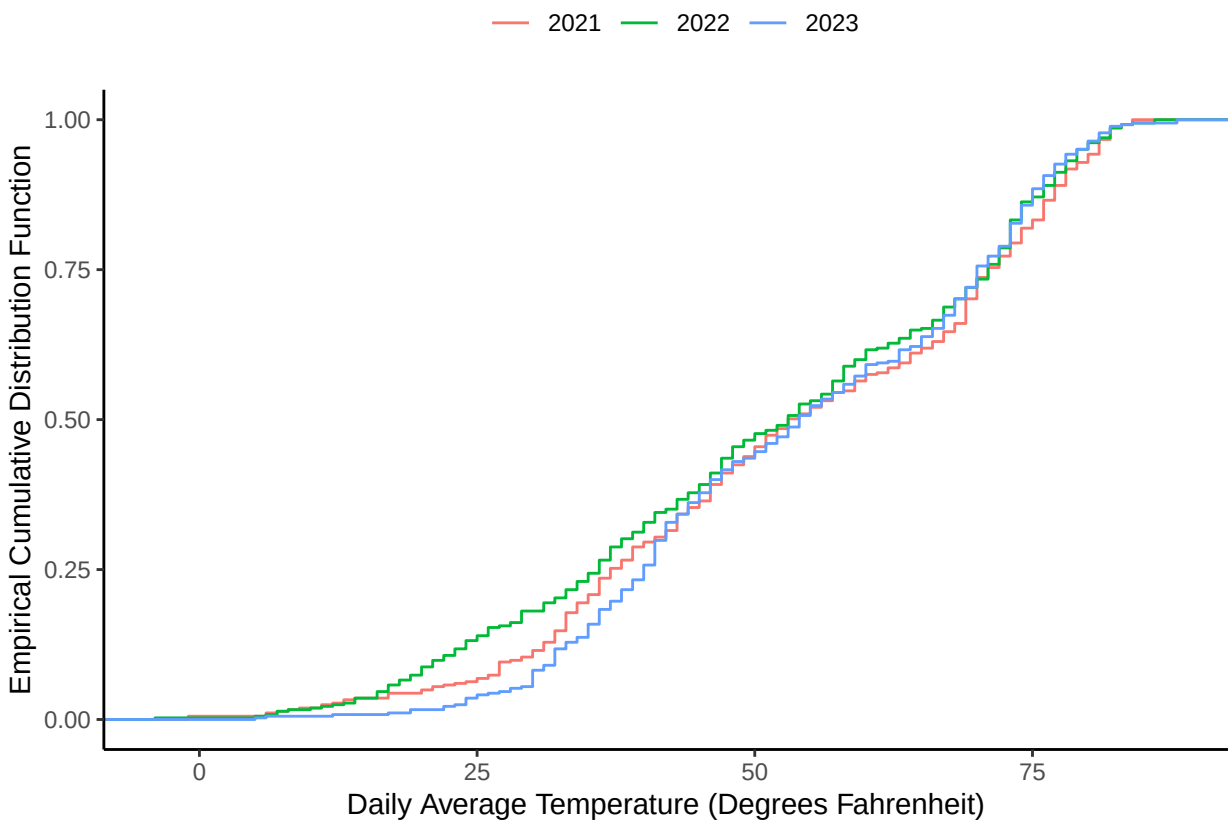
The annual temperature distribution for Chicago is **bimodal** — with one mode near 25–35°F (winter cluster) and another near 70–80°F (summer cluster), separated by a trough around 50–55°F. This is not consistent with normality. The conjecture is that temperatures in continental climates like Chicago spend *more time* at seasonal extremes (winter cold, summer heat) than at the spring/fall midpoints, because synoptic weather patterns are "sticky" — cold air masses and heat domes persist for weeks. The by-month facet confirms this: each individual month's temperature PDF is approximately unimodal and roughly normal, but aggregating across months creates the bimodal annual shape. This is important for LMP modeling because the **nonlinear relationship between temperature and load** (mirrored in HDD/CDD) means that the tails of the temperature distribution drive disproportionate price spikes (Week 1, Mordecai: "weather and grid load activity — load versus temperature conditioned on humidity"; Week 1: "Threshold Price Inelasticities: Volatile Conditions with Discontinuities and Disparities").

24. The following code produces yearly ECDFs of average daily temperature and average daily RT LMP for Chicago. What do you notice about the relationship, in terms of FOSD, between ECDFs for the LMPs? Does the same relationship hold for temperature? Conjecture as to why temperature patterns may or may not have translated to LMP patterns. Try the same exercise, comparing by month.

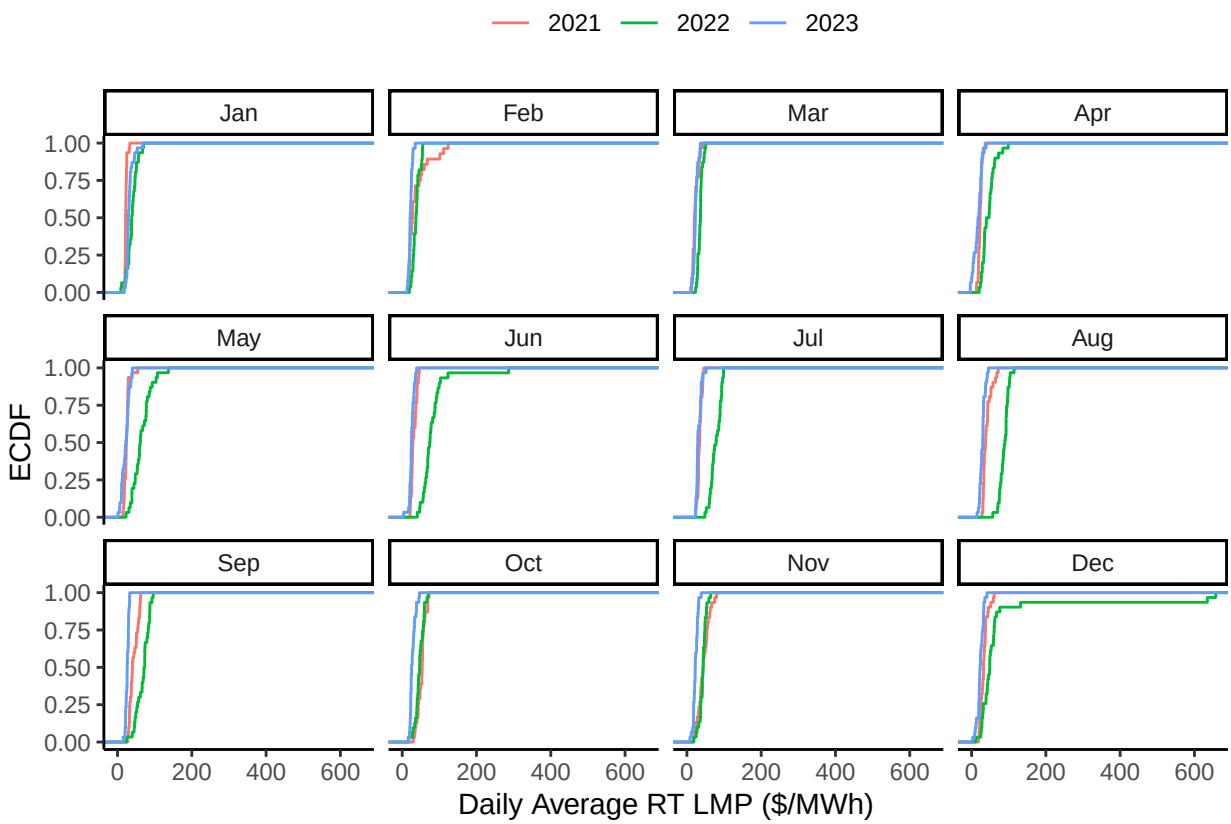
```
lmp_temp_Chicago <- data_Chicago[, .(
  mean_lmp_rt = mean(total_lmp_rt, na.rm = TRUE),
  mean_lmp_da = mean(total_lmp_da, na.rm = TRUE),
  vol_spread  = sd(spread, na.rm = TRUE),
  temp        = mean(avg_temperature_degrees_fahrenheit, na.rm = TRUE),
  humidity    = mean(relative_humidity_percent, na.rm = TRUE)
), by = .(date, month, year)]

# LMP ECDF by year
ggplot(lmp_temp_Chicago, aes(mean_lmp_rt, color = factor(year))) +
  stat_ecdf() +
  labs(x = 'Daily Average RT LMP ($/MWh)',
       y = 'Empirical Cumulative Distribution Function', color = '') +
  theme_classic() +
  theme(legend.position = 'top')
```




```r
# Temperature ECDF by year
ggplot(lmp_temp_Chicago, aes(temp, color = factor(year))) +
  stat_ecdf() +
  labs(x = 'Daily Average Temperature (Degrees Fahrenheit)',
       y = 'Empirical Cumulative Distribution Function', color = '') +
  theme_classic() +
  theme(legend.position = 'top')
```




```r
# By month - LMP
lmp_temp_Chicago$month <- factor(lmp_temp_Chicago$month, levels = month.abb)
ggplot(lmp_temp_Chicago, aes(mean_lmp_rt, color = factor(year))) +
  stat_ecdf() +
  labs(x = 'Daily Average RT LMP ($/MWh)',
       y = 'ECDF', color = '') +
  theme_classic() +
  theme(legend.position = 'top') +
  facet_wrap(~month)
```

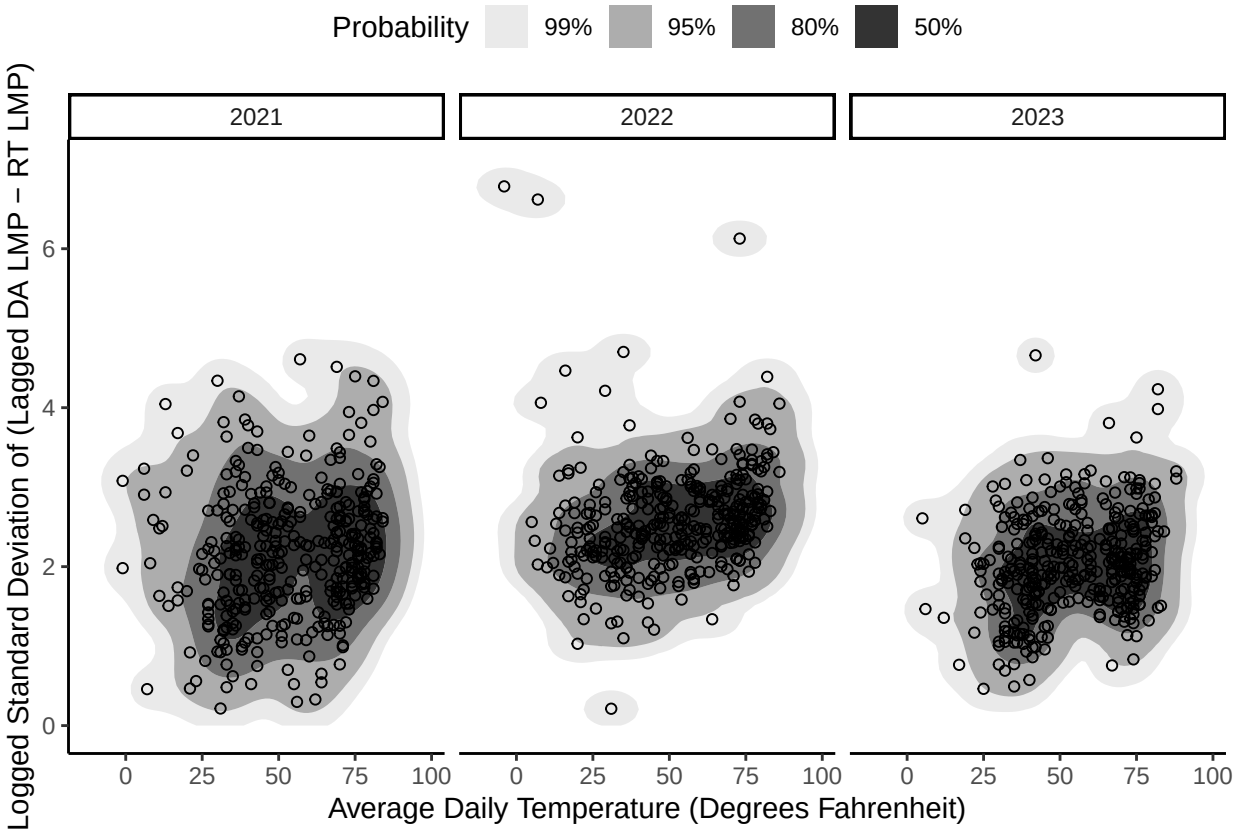


For LMPs: the 2022 ECDF lies clearly to the right of both 2021 and 2023 — 2021 and 2023 jointly first-order stochastically dominate 2022. This confirms that 2022 RT LMPs were uniformly higher (at virtually every quantile) than the adjacent years. **For temperature**: the ECDFs for 2021, 2022, and 2023 are tightly clustered and largely overlapping — no strong FOSD relationship holds. Year-to-year temperature variation in Chicago is small relative to the seasonal range (~60–65°F swing), so annual ECDFs look nearly identical.

The disconnect between temperature FOSD and LMP FOSD is fundamental: the mapping from temperature to LMP is **highly nonlinear**. Temperature drives demand through the merit order — at moderate temperatures demand is served by baseload (cheap) units, but as temperature extremes push demand onto peaking gas turbines, marginal cost jumps sharply. The 2022 LMP premium was primarily driven by the **gas price shock** (fuel input cost rather than a temperature anomaly), illustrating that LMPs reflect both weather-driven demand *and* supply-side cost dynamics simultaneously. The by-month comparison reveals that the 2022 premium was most pronounced in the summer months, consistent with gas peakers setting the LMP during heat-driven peak demand — a direct application of the merit order / spark spread concept (Week 1, Mordecai: "Spark spread dynamics -> Contracts-for-Differences (CfDs) -> Real Options Analysis"; Week 2, Birge: "On average, DA prices are actually higher than expected RT prices… consistent with market power of generators withholding supply in the DA market").

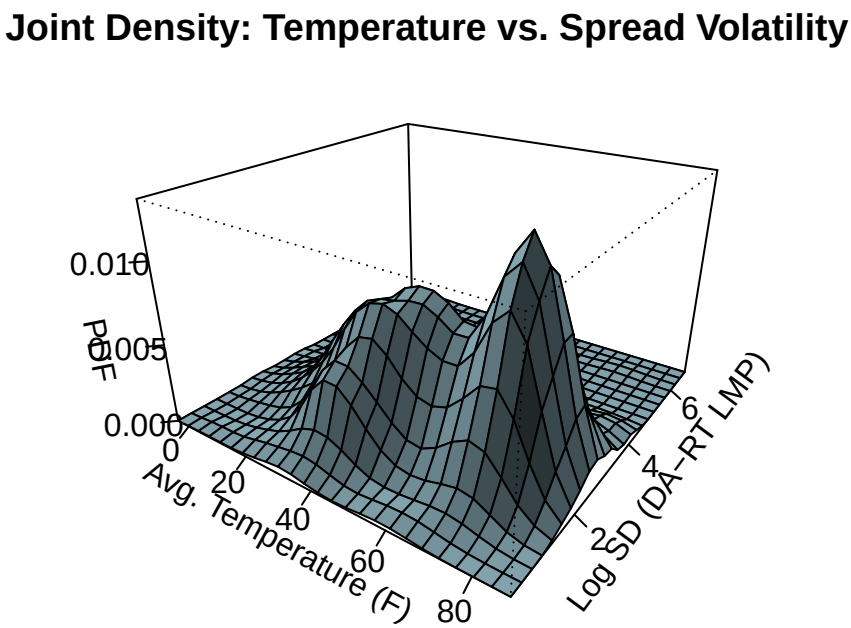
25. The code below plots the joint density of average daily temperature and the logged standard deviation of the daily volatility of the RT-DA LMP spread. Is there an obvious pattern thatemerges? Try identifying notable outliers in the lmp_temp_Chicago dataset. Conjecture as towhy outliers emerged on those days.

```
ggplot(lmp_temp_Chicago, aes(x=temp, y=log(vol_spread)) ) +geom_hdr(xlim = c(-25, 100), ylim = c(0,10)) +geom_po
```



26. The code below renders the plots above using a 3D density plot. Give it a try! What do thepeaks and valleys represent?

```
dens_Chicago <- kde2d(
  lmp_temp_Chicago[date > '2021-01-01']$temp,
  log(lmp_temp_Chicago[date > '2021-01-01']$vol_spread)
)

persp(dens_Chicago$x, dens_Chicago$y, dens_Chicago$z,
      xlab = "Avg. Temperature (F)",
      ylab = "Log SD (DA-RT LMP)",
      zlab = "PDF",
      theta = 35, phi = 25, expand = 0.6,
      col = "lightblue", shade = 0.4, ticktype = "detailed",
      main = "Joint Density: Temperature vs. Spread Volatility")
```

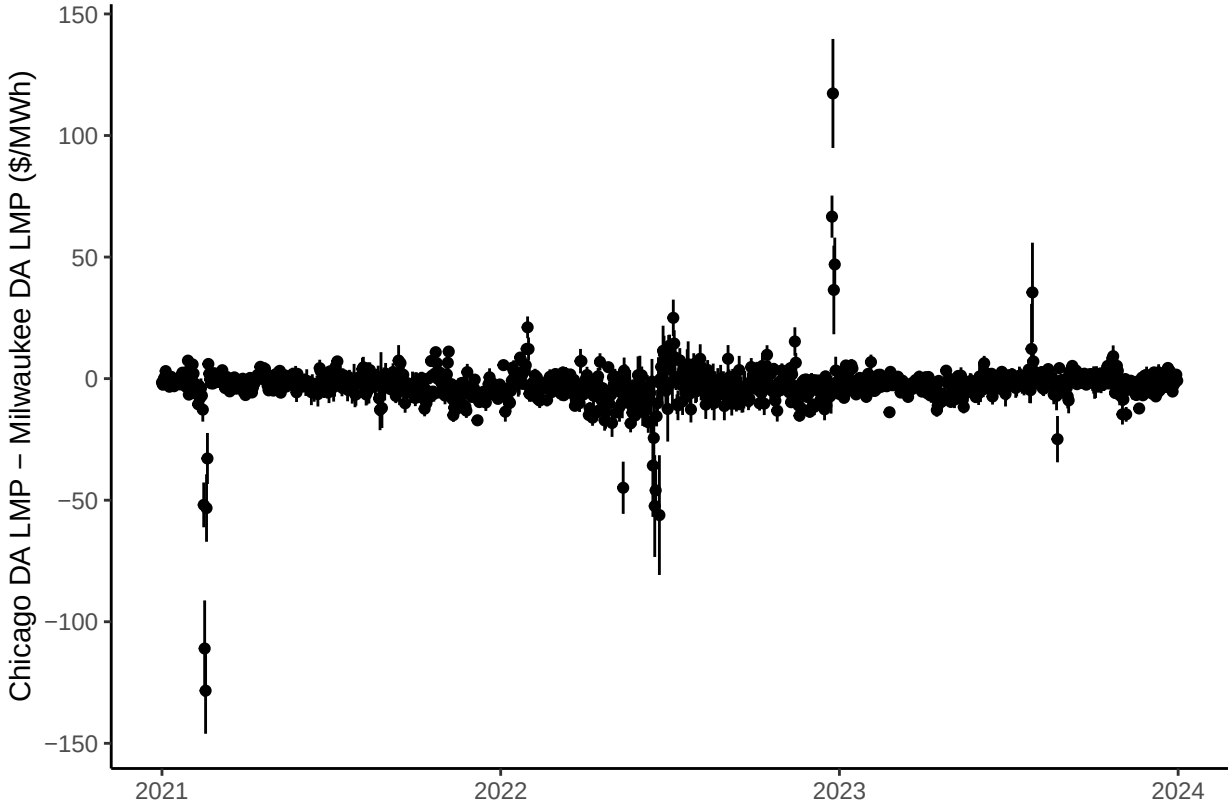


VII. Comparing Interconnected Transmission Regions

The states west of PJM belong to the Midcontinent Independent System Operator (MISO). Milwaukee and Chicago, two cities merely 90 minutes apart by car, belong to two different ISOs(Milwaukee in MISO and Chicago in PJM). In this section, we explore differences in LMPs and their responsiveness to weather patterns, in two separate (yet interconnected) parts of the grid.

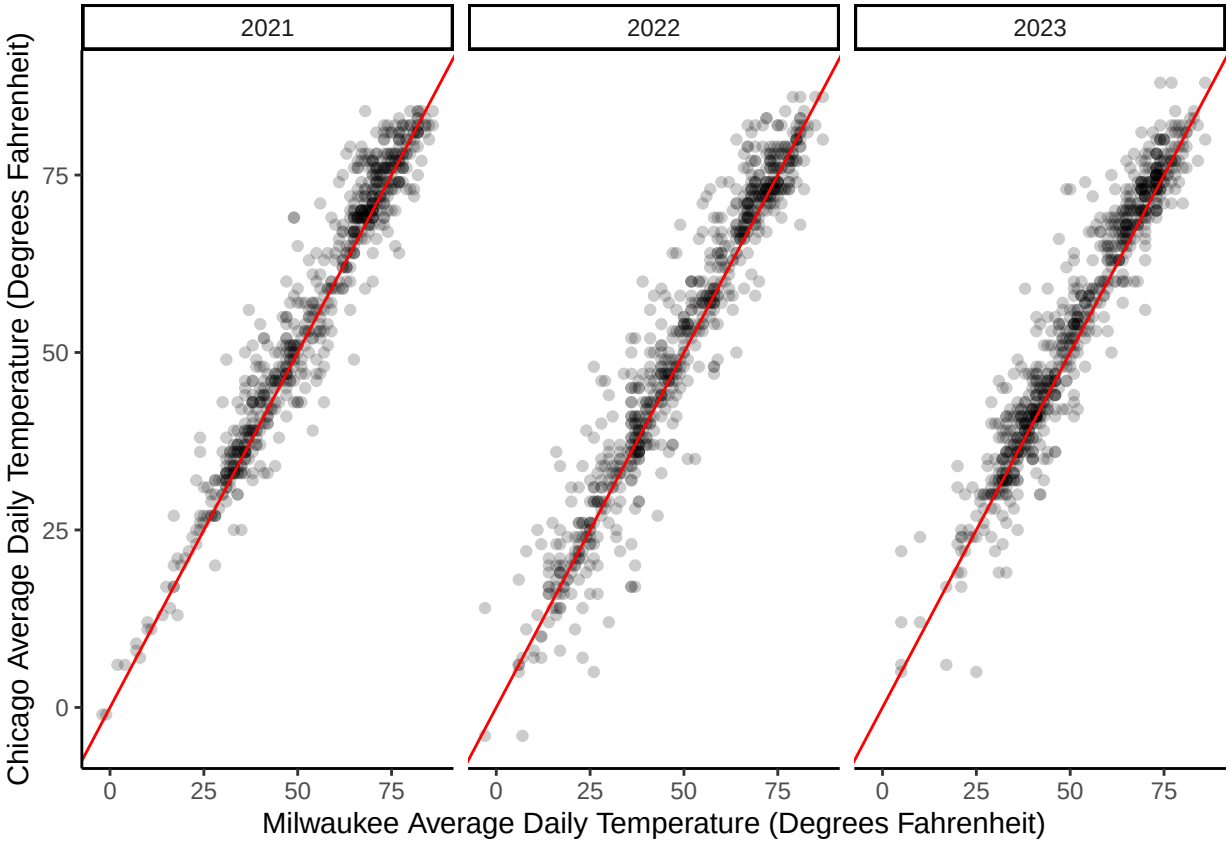
27. The code below plots the spread between average daily DA LMPs in Chicago and Oak Creek Power Plant (near Milwaukee). Consider the outlier days. Could they be explained byvariations in weather? Plot the same figure for the spread in average daily temperatures totest your hypothesis.

```
lmp_cities <- data.table(date = data_Milwaukee$date,Milwaukee = data_Milwaukee$DA_LMP,Chicago = data_Chicago$tot

ggplot(lmp_cities, aes(x=date,y=da_spread)) +stat_summary(size=0.2) +labs(x='',y='Chicago DA LMP - Milwaukee DA
```



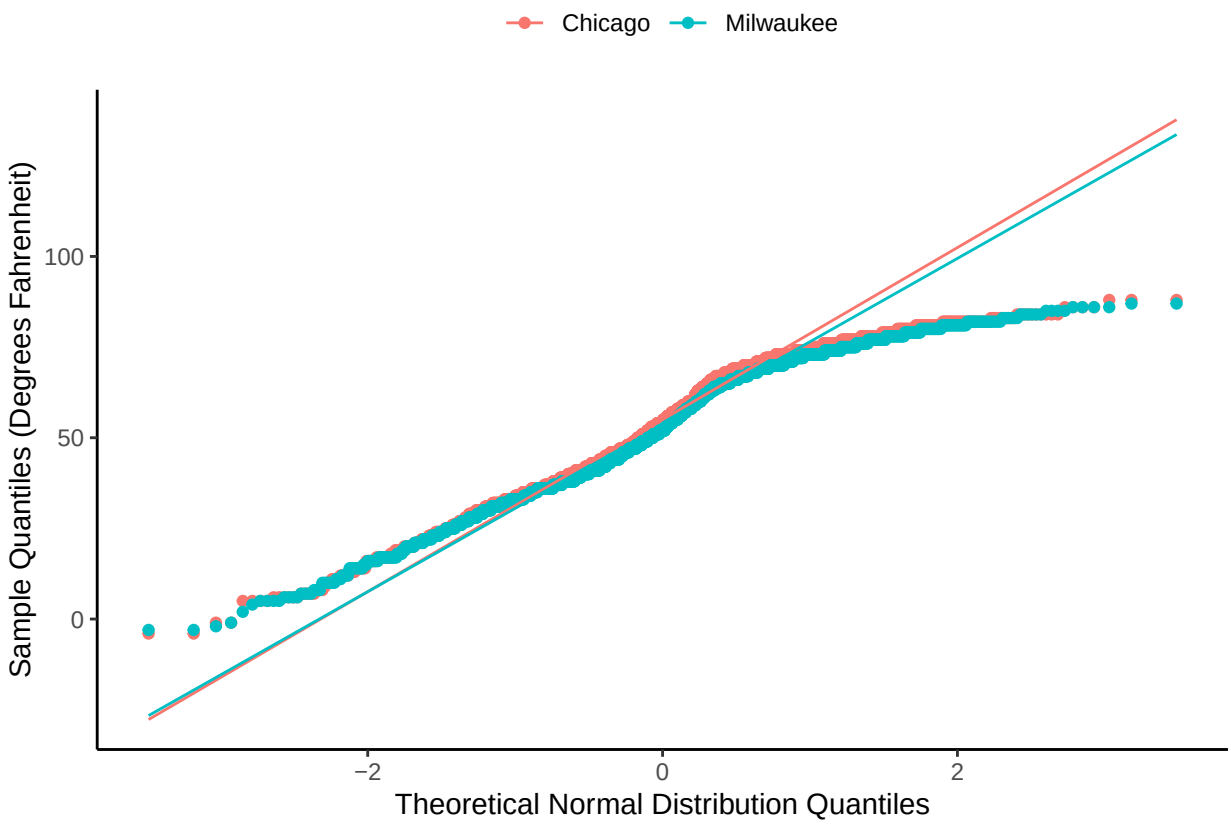
28. One might expect that weather in Milwaukee and Chicago to be similar. The code below generates scatter plots comparing average daily temperatures in Milwaukee and Chicago, showing temperatures in the two cities are closely aligned. Try creating a similar plot foraverage daily DA LMPs in the two cities. How closely aligned are the average daily DALMPs in the two cities? Why might LMPs in the two cities differ more than temperatures

```
temperature_cities <- data.table(
  date      = data_Milwaukee$date,
  Milwaukee = data_Milwaukee$avg_temp_f,
  Chicago   = data_Chicago$avg_temperature_degrees_fahrenheit
) %>% unique()

ggplot(temperature_cities, aes(x = Milwaukee, y = Chicago)) +
  geom_point(alpha = 0.2) +
  geom_abline(intercept = 0, slope = 1, color = 'red') +
  labs(x = 'Milwaukee Average Daily Temperature (Degrees Fahrenheit)',
       y = 'Chicago Average Daily Temperature (Degrees Fahrenheit)',
       color = '') +
  facet_wrap(~year(date)) +
  theme_classic()
```



29. We can examine whether daily temperatures in Chicago and Milwaukee are drawn from a normal distribution using a Q-Q plot. Based on the Q-Q plot created from the code below, towhat extent are these temperature series normal? Try creating a similar Q-Q plot comparingAustin and Corpus Christi temperatures to the normal distribution. As an optional step, trycreating a Q-Q plot of Chicago average temperatures against Milwaukee temperatures. Arethese series drawn from the same distribution?

```
ggplot(temperature_cities %>% pivot_longer(!date, names_to = 'series', values_to ='temp'),aes(sample = temp, col
```

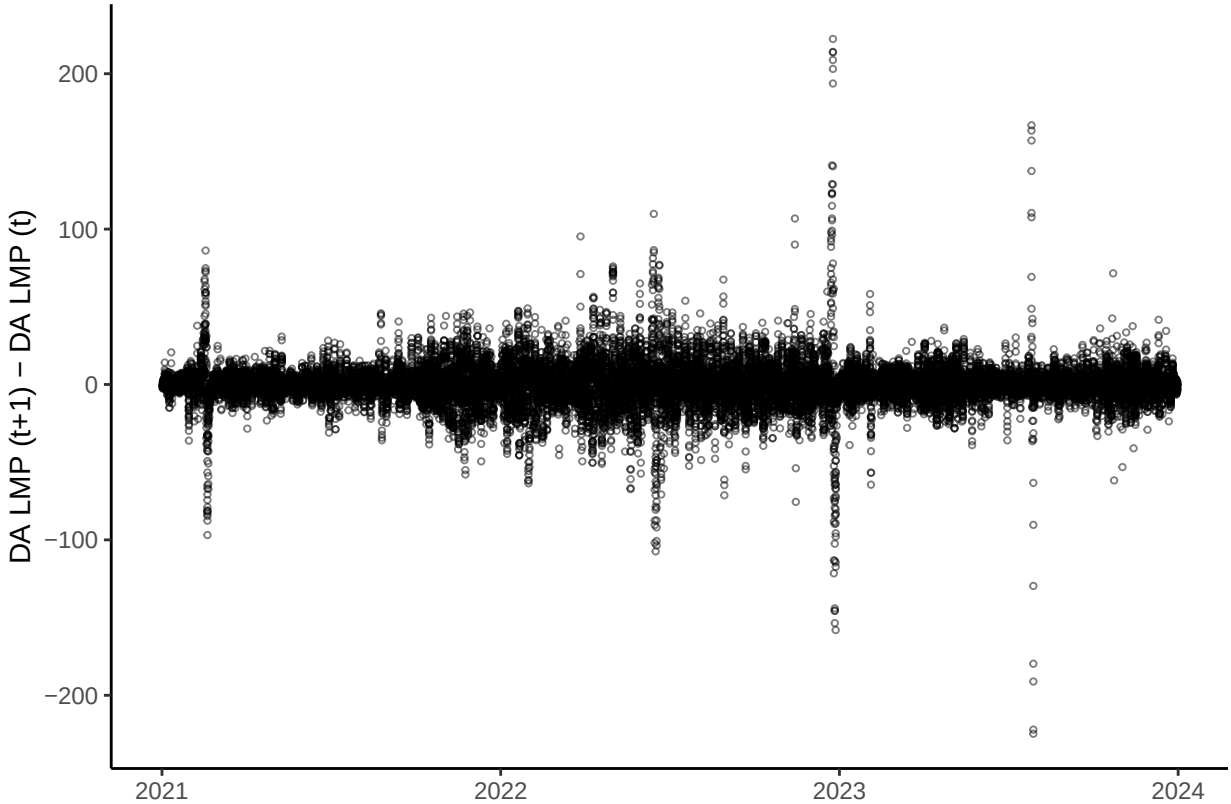


30. In February 2021, Texas suffered a devastating winter storm that exacerbated the vulnerabilities of its power grid. The code below displays average daily temperatures and real-time LMPs in Chicago, Milwaukee, and Corpus Christi during this month. Note the scalefor each plot. What do you observe from these plots? Conjecture as to the relationshipbetween prices and weather. Do you think the relationship is linear? Describe the role andnuances of anomalous weather.

VIII. Contracts for Differences

31. The code below plots day-over-day changes in the Chicago hourly DA LMP. What does eachpoint on the plot represent? Why are the observations on for most days approximatelysymmetric around zero? Which days exhibited the largest day-over-day changes in DA LMP?How do day-to-day changes in DA LMP relate to contracts for differences (CfDs)?

```
ggplot(data_Chicago[order(date)] %>%.[,prev_lmp_da := shift(total_lmp_da, type = 'lag'), by = hour],aes(x = date
```



32. Form your own hypotheses about how day-over-day changes in DA LMP may vary accordingto different dimension of the Chicago data (e.g., day-of-week, temperature changes, etc.).Test them. Discuss how patterns in day-over-day changes in DA LMP may relate to energystorage needs.

33. The code below creates a data set containing the ratio of day-over-day DA LMP spreads between Chicago and Milwaukee. That is, it computes the day-over-day change in hourly DA LMP for Chicago and Milwaukee and then computes their ratio in the spread_ratio field. Use this data set to plot spread_ratio and interpret the magnitude of spread_ratio. What do larger values mean and how do they relate to incentives for energy transmission and storage given your interpretation, what has changed in 2023 relative to 2021 and 2022?

```
internodal_spread <- data.table(date = data_Chicago$date,hour = data_Chicago$hour,day_of_week = data_Chicago$day
```

34. In the previous question, we explored the ratio of day-over-day DA LMP spreads between Milwaukee and Chicago. The code below creates a similar dataset for the ratio of day-overday DA LMP spreads between Chicago and Corpus Christi, TX; the spread_ratio_ercot variable holds this data (Corpus Christi belongs to the Electric Reliability Council of Texas,or "ERCOT"). What do larger values mean, and how do they relate to incentives for energy transmission and storage?

```
internodal_spread$spread_ratio_ercot <- internodal_spread$Chicago_da_spread /internodal_spread$CorpusChristi_da_
```

35. The code below tells you the number of days that spread_ratio and spread_ratio_ercot were above 1, respectively, between 2021 and 2023. Interpret these results.

```
count1 <- sum(internodal_spread$spread_ratio > 1, na.rm = TRUE)
count2 <- sum(internodal_spread$spread_ratio_ercot > 1, na.rm = TRUE)
print(count1)
```

```
## [1] 8860
```

```r
print(count2)
```

```
## [1] 6082
```

```r
count3 <- sum(internodal_spread$spread_ratio > 0, na.rm = TRUE)
count4 <- sum(internodal_spread$spread_ratio < 0, na.rm = TRUE)
print(count3)
```

```
## [1] 16392
```

```r
print(count4)
```

```
## [1] 9792
```

36. Construct a spread metric that considers the contemporaneous spread between Chicago and Milwaukee to those from the preceding day. Interpret this ratio. How does it differ `fromspread_ratio`? Try comparing these two different series using a plot.

IX. Load and Price Duration Curves

37. Use the code below to load a dataset, created by Bolarinwa Ajanaku,23 containing daily observations of PJM data from January 1, 2016 to December 31, 2019.24 This dataset includes a number of series, including load, RT LMP, as well as supply and prices for natural gas and wind.

```r
daily_PJM <- fread(paste0(wd, "/", "PJM Daily.csv"))
```

38. The code below generates annual load duration curves for PJM, over the years 2016-2019.Based on the plot, which years featured higher demand for electricity? Try adapting the codeto annual price duration curves for PJM, using the rtm_price variable in place ofActual_load_GWh. Which year featured the highest LMP prices? Is this consistent with yourinference based on the load duration curve?

```r
# ldc <- daily_PJM[, .(Date, Actual_load_GWh)] %>%.[, Date := as.Date(Date, format = "%m/%d/%Y")] %>%.[, rank :=
#
# ggplot(ldc, aes(rank, Actual_load_GWh, color = factor(year(Date)))) +geom_line() +scale_x_continuous(expand =
```

39. Try plotting load duration curves for Austin, Milwaukee, and Corpus Christi, using the"load_mw" field. Which cities typically have higher loads?